

TD Trouan

1)

Problèmes rencontrés :

a) On ne doit pas inscrire d'enfants à une prestation au mois d'août. Cependant, cet été cela s'est produit.

b) Les enfants qu'on avait inscrit à la 1ère journée "Escalade" du 9/10/2024 n'ont pas pu être inscrits à la 2ème journée "Escalade" le 16/10/2024 alors qu'il restait des places et c'est le cas quelque soit l'activité, si un enfant avait déjà été inscrit à cette prestation.

c) On a constaté que le nombre d'enfants inscrits à la journée "Découverte sport nautique" était supérieur au nombre de places initialement prévues. Cela pose évidemment un problème d'encadrement des enfants.

Solution à mettre en place :

a) Ajouter une vérification pour bloquer toute tentative d'inscription pour les dates comprises entre le 1er et le 31 août.

Avec une contrainte CHECK ou une procédure stockée lors de l'insertion.

b) Créer une règle ou procédure stockée qui vérifie si un enfant est déjà inscrit à une activité donnée. Puis permettre une inscription à une autre session de la même activité.

c) Ajouter un trigger sur la table d'inscription pour vérifier :
Le nombre d'inscriptions existantes pour une activité donnée.

Comparer ce nombre avec le nombre de places disponibles pour cette activité.

Si le nombre d'inscriptions dépasse la capacité, déclencher une erreur et annuler l'opération.

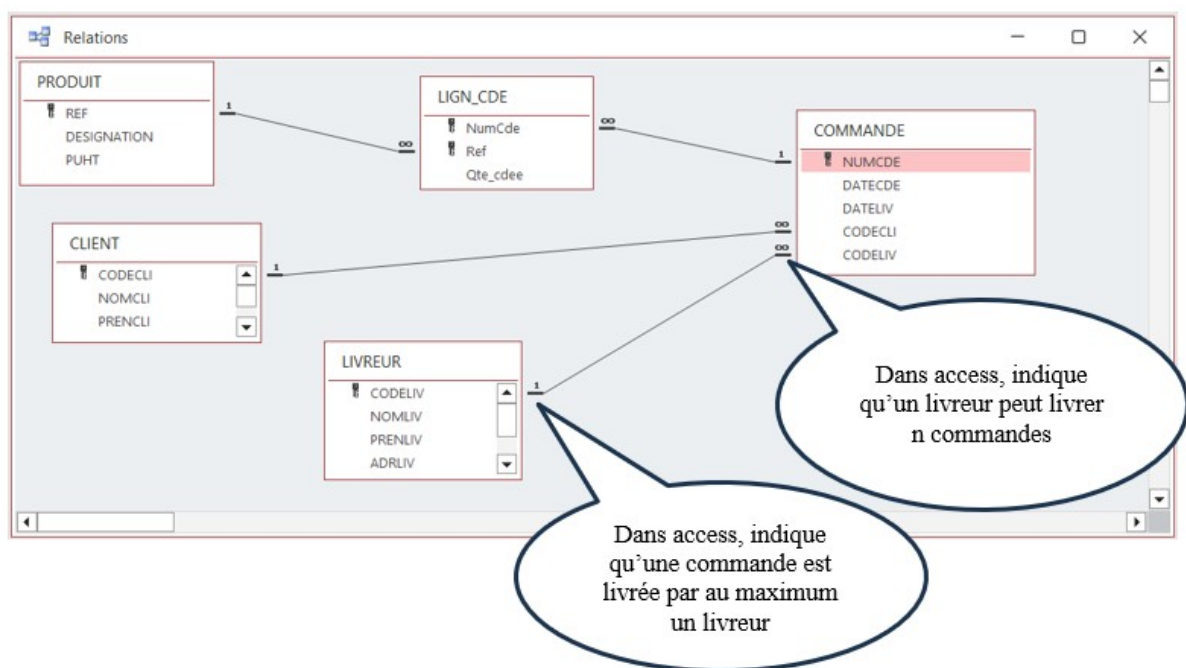
SGBD TP n°1 Picoło.

Ce TP utilise beaucoup de notions déjà abordées en SIO1. Le support de cours SQL de l'an dernier vous est fourni, n'hésitez pas à la consulter en faisant par exemple des recherches de mots (grâce à CTRL + F).

L'entreprise Picoło qui vend des surgelés à des particuliers (les commandes sont livrées) possède une base de données Access pour enregistrer les commandes des clients ainsi que les livraisons effectuées.

Elle a demandé à un étudiant en stage de transposer cette base de données au format MariaDB.

Voici le schéma de la base de données sous Access :



Le stagiaire a utilisé un logiciel nommé Bullzip MS Access to MSSQL qui lui a permis de se connecter à la base de données Access et de générer un script SQL. Ce script lui a demandé un gros travail d'adaptation car la syntaxe n'était pas correcte pour MariaDB.

A la fin de son stage, il a fourni à l'entreprise un script SQL qui permet de créer la BD avec un jeu d'essai mais il n'a pas pu faire davantage.

Ce TP consiste maintenant à réaliser des requêtes de test. Dans ce document, vous devrez réaliser un copier-coller des requêtes réalisées qui répondent au besoin exprimé. Vous sauvegarderez ce document récapitulatif de votre travail dans un dossier TP N°1 Picoło dans un dossier Bloc 2 SGBD MJ dans votre espace de stockage.

☞ Sous MariaDB, créer une BD nommée VotreNom_Picoło : le fait de préfixer le nom de vos BD par votreNom, permettra à PHPMyadmin d'afficher vos bases de données de manière regroupée.

Fait.

☞ Créer le contenu de la base de données à l'aide du script fourni.

Fait.

☞ Réaliser les requêtes SQL suivantes :

1. Affichez la liste des clients (Code, nom, prénom, adresse complète) qui ont passé une commande le 19/01/2024 (triée par ordre alphabétique). Format de date MySQL : 'aaaa-mm-jj' à la place de 'jj/mm/aaaa'.

Réponse vous devez obtenir 4 clients : Stéphane BONNIN, Nicole CORINNE, Marc DURAND, et Nathalie MULLER.

SELECT 'CODECLI', 'NOMCLI', 'PRENCLI', 'ADRCLI' FROM 'client' NATURAL JOIN commande WHERE DATECDE = '2024-01-19' ORDER BY 'NOMCLI';

2. Affichez la liste des clients (Code, nom, prénom, adresse complète) qui ont passé une commande en janvier 2024 (triée par ordre alphabétique).

Réponse vous devez obtenir 5 clients : Stéphane BONNIN, Nicole CORINNE, Marc DURAND, Delphine Julien et Nathalie MULLER car il faut penser à supprimer les doublons avec DISTINCT car un client peut commander plusieurs fois en janvier 2024.

SELECT DISTINCT client.CODECLI, 'NOMCLI', 'PRENCLI', 'ADRCLI' FROM 'client' INNER JOIN commande ON commande.CODECLI = client.CODECLI WHERE DATECDE LIKE '2024-01-%' ORDER BY 'NOMCLI';

3. Quel est le contenu de la commande n°10 ? affichez pour chaque article commandé, sa désignation, son prix, la quantité commandée et le total pour cet article. Amélioration : Vous devez présenter le total arrondi avec uniquement 2 décimales en utilisant la fonction ROUND().

SELECT produit.ref, produit.DESIGNATION, produit.PUHT, lign_cde.Qte_cdee, Round(produit.PUHT*lign_cde.Qte_cdee, 2) FROM 'commande' INNER JOIN lign_cde ON lign_cde.NumCde = commande.NUMCDE INNER JOIN produit ON produit.REF = lign_cde.Ref WHERE commande.NUMCDE = 10;

Réponse :

ref	designation	puht	qte_cdee	total
1277	Vacherin vanille/café - 12 parts	10.6724	12	128.07
1279	Cônes vanille (x6)	2.7443	1	2.74

4. Avec une requête SQL, ajouter un article à cette commande : l'article avec la référence 1284 commandé en 3 exemplaires.

INSERT INTO 'lign_cde'('NumCde', 'Ref', 'Qte_cdee') VALUES ('10','1284','3');

5. Utiliser la requête 3 pour vérifier le nouveau contenu de la commande n°10 ? que constatez-vous ? pourquoi ? Ecrire les requêtes SQL qui permettent de corriger ce problème.

Il n'y pas le nouvel article car il n'a pas été rajouter dans la table produit.

INSERT INTO 'produit'('REF', 'DESIGNATION', 'PUHT') VALUES ('1284','Cônes carâmel','15')

6. Vérifier les liens entre les tables de votre base de données grâce au concepteur de PHPMyadmin. Si toutes les tables ne sont pas reliées, vous avez mal traité la question 5 qui devait attirer votre attention sur le fait qu'une base de données sans foreign Key peut donner lieu à des incohérences comme un produit 1284 ajouté à la question 4 dans lign_cde alors qu'il n'existe pas dans produit !

Non reliée.

7. Si toutes les tables de votre BD ne sont pas reliées, vous devez écrire les requêtes SQL pour ajouter les contraintes de clés étrangères !

ALTER TABLE commande ADD CONSTRAINT `FK\_commande\_client` FOREIGN KEY (`CODECLI`) REFERENCES client (`CODECLI`);

ALTER TABLE commande ADD CONSTRAINT `FK\_commande\_livreur` FOREIGN KEY (`CODELIV`) REFERENCES livreur (`CODELIV`);

ALTER TABLE lign_cde ADD CONSTRAINT `FK\_ligncde\_commande` FOREIGN KEY (`NumCde`) REFERENCES commande (`NUMCDE`);

ALTER TABLE lign_cde ADD CONSTRAINT `FK\_ligncde\_produit` FOREIGN KEY (`Ref`) REFERENCES produit (`REF`);

Fait.

8. Quel est le prix du produit le moins cher ?

SELECT MIN(`PUHT`) FROM `produit`; **résultat : 1,6251**

9. Quel est le produit qui coûte le moins cher ? Aide : Il vous faut créer une requête avec une sous-requête (celle qui affiche le prix du produit le moins cher).

SELECT `PUHT` FROM `produit` WHERE `PUHT` = (SELECT MIN(`PUHT`) FROM `produit`); **résultat : 1,6251**

10. Ecrire la requête qui liste les clients (triés alphabétiquement) n'ayant pas commandé. Aide : INNER JOIN éliminerait ces clients (pour lesquels on ne trouve pas de ligne avec client.codeCli=commande.codeCli).

SELECT `NOMCLI`, `PRENCLI`, client.`CODECLI` FROM `client` WHERE client.`CODECLI` NOT IN (SELECT CODECLI FROM commande) ORDER BY `NOMCLI`;

11. A partir de la requête n°3, créer une requête pour afficher chaque commande avec son contenu (numcde, ref, prix, qte, total de la ligne). Copier le résultat de la requête :

SELECT commande.NUMCDE, produit.ref, produit.PUHT, lign_cde.Qte_cdee, Round(produit.PUHT*lign_cde.Qte_cdee, 2) FROM `commande` INNER JOIN lign_cde ON lign_cde.NumCde = commande.NUMCDE INNER JOIN produit ON produit.REF = lign_cde.Ref ORDER BY `NUMCDE`;

NUMCDE ▲ 1	ref	PUHT	Qte_cdee	Round(produit.PUHT*lign_cde.Qte_cdee, 2)
1	1250	1.6251	56	91.01
1	1273	6.4034	31	198.51
1	1275	6.5559	54	354.02
1	1280	3.0492	11	33.54
2	1274	6.7083	32	214.67
2	1281	2.7443	41	112.52
3	1273	6.4034	2	12.81
4	1280	3.0492	1	3.05
5	1250	1.6251	53	86.13
5	1279	2.7443	63	172.89
5	1280	3.0492	13	39.64
6	1273	6.4034	1	6.40
6	1275	6.5559	2	13.11
6	1277	10.6724	34	362.86
6	1280	3.0492	2	6.10
7	1277	10.6724	1	10.67
8	1250	1.6251	2	3.25
9	1273	6.4034	2	12.81
9	1283	3.6591	3	10.98
10	1277	10.6724	12	128.07
10	1279	2.7443	1	2.74
10	1284	15	3	45.00

Remarque : la table commande n'est pas utile car numCde est dans lign_cde et nous n'avons pas besoin de dateCde.

Que devez-vous faire si je vous demande de calculer manuellement (sans requête) le total de chaque commande ? **Je sais pas.**

☞ Consulter le site W3schools pour réviser la clause GROUP BY : https://www.w3schools.com/sql/sql_groupby.asp (si vous souhaitez plus de détails, vous pouvez consulter dans Bloc2SGBD\Rappes_SQL SIO1, le fichier SQL_GROUP BY exemples, ainsi que le cours LangageSQL2 - LMD – Début.

12. A partir de la requête n°7, créer une requête qui calcule le total de chaque commande (en faisant le cumul du total de chaque ligne de commande pour une même commande). On souhaite les numéros de commande et leur total. Vous arrondirez le cumul à la fin pour améliorer.

SELECT commande.NUMCDE, SUM(Round(produit.PUHT*lign_cde.Qte_cdee, 2)) FROM `commande` INNER JOIN lign_cde ON lign_cde.NumCde = commande.NUMCDE INNER JOIN produit ON produit.REF = lign_cde.Ref GROUP BY commande.`NUMCDE` ORDER BY `NUMCDE`;

13. Créer une vue TotalCommandes alimentée par la requête n°12. Testez le résultat fourni par la vue.

```
CREATE VIEW TotalCommandes (numero, totalcde)
AS SELECT commande.NUMCDE, SUM(Round(produit.PUHT*lign_cde.Qte_cdee, 2))
FROM `commande`
INNER JOIN lign_cde
ON lign_cde.NumCde = commande.NUMCDE
INNER JOIN produit
ON produit.REF = lign_cde.Ref
GROUP BY commande.`NUMCDE`
ORDER BY `NUMCDE`
```

Le résultat de la vue est le même résultat que la requête n°12.

14. Avec une requête SQL, indiquez quelles sont les commandes d'un total supérieur à 300€ ?

SELECT * FROM `totalcommandes` WHERE totalcde > 300;

15. Commandes passées le même jour que la commande n°6.

```
SELECT `NUMCDE`, `DATECDE` FROM `commande`
WHERE `DATECDE` = (SELECT `DATECDE` FROM commande WHERE NUMCDE = 6);
```

16. Nombre de commandes effectuées en janvier 2024 pour chaque produit.

SELECT produit.REF, `DATECDE`, COUNT(lign_cde.Qte_cdee) FROM `commande` INNER JOIN lign_cde ON commande.NUMCDE = lign_cde.NumCde INNER JOIN produit on lign_cde.Ref = produit.REF WHERE `DATECDE` LIKE '2024-01-%%' GROUP BY produit.REF;

17. Clients n'ayant jamais commandé de cônes (démarche : on doit trouver les clients ayant commandé des cônes, et ensuite demander à voir les clients qui n'en font pas partie).

```
SELECT `NOMCLI`, `PRENCLI`, client.`CODECLI` FROM client
WHERE commande.NUMCDE NOT IN (SELECT lign_cde.NumCde FROM commande
INNER JOIN commande
on client.CODECLI = commande.CODECLI
INNER JOIN lign_cde
ON commande.NUMCDE = lign_cde.NumCde
INNER JOIN produit
ON lign_cde.Ref = produit.REF WHERE produit.DESIGNATION LIKE 'Cone%')
GROUP BY `CODECLI`
```

18. CA par mois pour 2024 : vous devez utiliser les fonctions YEAR() et MONTH() (étape intermédiaire afficher les numéros de commande de 2024 avec le mois de la commande).

```
SELECT MONTH ('DATECDE'), SUM(Round(produit.PUHT*lign_cde.Qte_cdee, 2)) FROM  
'commande' INNER JOIN lign_cde ON lign_cde.NumCde = commande.NUMCDE INNER JO  
IN produit ON produit.REF = lign_cde.Ref WHERE YEAR ('DATECDE') = 2024 GROUP B  
Y MONTH ('DATECDE');
```

Pour les plus avancés :

19. Ecrire la requête qui affiche le meilleur client (si 2 clients sont à égalité, ils doivent apparaître tous les deux).

SGBD TP n°2 - Limovideo

Première partie : Création et test de la base de données

Le vidéoclub LIMOVVIDEO loue des DVD à la journée. Ses clients sont des particuliers, des associations et des comités d'entreprise. Pour emprunter, ils doivent au préalable payer un abonnement. Le vidéoclub vous a demandé de poursuivre l'analyse débutée par un étudiant de BTS SIO.

Voici le dictionnaire des données épuré qu'il a établi :

Désignation	Code	Type
Code du type client	T_code	Entier
Intitulé du type client	T_Intitulé	chaîne(22)
Taux TVA	TVA	Float (3,2)
Numéro du client	C_Numéro	Entier
Nom du client	C_Nom	chaîne (25)
Rue du client	C_Rue	chaîne (50)
Code postal du client	C_CP	chaîne (5)
Ville du client	C_Ville	chaîne (50)
Numéro DVD	D_Num	Entier
Date Achat DVD	D_DateAch	Date
Prix/jour	Prixj	Float(3,2)
Etat du DVD	D_Etat	"N", "B", "M"
Code du Film	F_Code	Entier
Titre du Film	F_Titre	chaîne (40)
Duree du Film	F_Duree	Entier
Coût Abonnement	Abo_Cout	Entier
Numéro de Genre	G_Num	Entier
Intitulé du Genre	G_Int	chaîne (25)
Date d'emprunt	Date_Emp	Date
Date de retour	Date_Ret	Date

Remarques : le prix par jour est de 1,5€ HT pour les comités d'entreprises (Code du type client=1), 1,80€ HT pour les associations (Code du type client=2) et 2€ HT pour les particuliers (Code du type client=3).

L'abonnement coûte 10€.

Etat d'un DVD: "N" pour "Neuf", "B" pour "Bon", "M" pour moyen

Un film peut être rattaché à plusieurs genres.

La TVA est la même pour tous les Dvds et égale à 20%.

On souhaite garder un historique des prêts donc les prêts même si tous les DVD sont retournés ne sont pas supprimés.

Les DVDs d'un prêt ne sont pas tous rapportés en même temps (il arrive qu'un client ramène seulement une partie des Dvds loués et garde le reste des Dvds).

Travail à faire (A restituer dans un compte-rendu détaillé) :

- 1) A partir des informations de la page précédente, dégager les entités nécessaires (trouvez les identifiants) pour la réalisation du MCD. **Fait.**
- 2) Éventuellement compléter le MCD (ajout de propriétés). **Fait.**
- 3) Trouvez les propriétés des entités (elles doivent être directement en dépendance fonctionnelle avec l'identifiant de l'entité). **Fait.**
- 4) Tracer le Modèle conceptuel des données avec historique des prêts avec l'outil JMerise (pour les chaînes de caractères, utiliser correctement les VARCHAR et les CHAR). **Fait.**
- 5) Justifier par 2 phrases les cardinalités d'une association matérialisant un lien UN_PLUSIEURS. **Un client peut emprunter plusieurs DVD.**
Plusieurs DVD peuvent être emprunter par un client.
- 6) Justifier par 2 phrases les cardinalités d'une association matérialisant un lien PLUSIEURS_PLUSIEURS. **Plusieurs client peuvent faire plusieurs emprunt.**
Plusieurs emprunt peuvent être fait par plusieurs client.
- 7) Un client peut-il emprunter à des dates différentes le même Dvd ? cela doit être possible : est-ce que votre BD le permet ?
Un client peut emprunter à des dates différentes le même Dvd. Fait
- 8) Un client peut-il emprunter plusieurs DVD lors d'un prêt. Peut-il les rapporter à des dates différentes ? Faites le nécessaire pour que cela soit possible.
Un client peut emprunter plusieurs DVD lors d'un prêt. Il peut les rapporter à des dates différentes. Fait
- 9) Faites valider votre MCD. **Fait.**
- 10) Générer le modèle relationnel et le script pour MariaDB. **Fait.**
- 11) Générer la base de données MariaDB correspondante. **Fait.**
- 12) Ajouter une contrainte pour contrôler l'état du DVD (voir les remarques en page 1).
ALTER TABLE dvd ADD CONSTRAINT CHK_dvd_D_Etat CHECK ('D_Etat' IN 'N','B','M'); **Fait.**
- 13) Peupler votre base de données (création de clients, de DVDs, de prêts, ajout de type_client,) de préférence en utilisant des requêtes SQL.

```
INSERT INTO `type_client`(`T_Code`, `T_Intitule`, `Prixj`) VALUES ('1','comités d'entreprises','1.5');
```

```
INSERT INTO `type_client`(`T_Code`, `T_Intitule`, `Prixj`) VALUES ('2','associations','1.8')
```

```
INSERT INTO `type_client`(`T_Code`, `T_Intitule`, `Prixj`) VALUES ('3','particuliers','2');
```

```
INSERT INTO `client`(`C_Numero`, `C_Nom`, `C_Rue`, `C_CP`, `C_Ville`, `T_Code`) VALUES ('1','Bernard','10 rue du peupliers','95000','Paris','1');
```

```
INSERT INTO `client`(`C_Numero`, `C_Nom`, `C_Rue`, `C_CP`, `C_Ville`, `T_Code`) VALUES ('2','Dupont','22 avenue des Champs Elysées','95000','Paris','2');
```

```
INSERT INTO `client`(`C_Numero`, `C_Nom`, `C_Rue`, `C_CP`, `C_Ville`, `T_Code`) VALUES ('3','Durand','7 rue des Lilas','95000','Paris','3');
```

```
INSERT INTO `film`(`F_Code`, `F_Titre`, `F_Duree`) VALUES ('1','Titanic','3');
```

```
INSERT INTO `film`(`F_Code`, `F_Titre`, `F_Duree`) VALUES ('2','Harry Potter et la coupe de feu','2');
```

```
INSERT INTO `film`(`F_Code`, `F_Titre`, `F_Duree`) VALUES ('3','Le Roi Lion','2');
```

```
ALTER TABLE dvd ADD CONSTRAINT CHK_dvd_D_Etat CHECK (D_Etat IN 'N','B','M');
```

```
INSERT INTO `dvd`(`D_Num`, `D_DateAch`, `D_Etat`, `F_Code`) VALUES ('1','2020-10-10','N','1')
INSERT INTO `dvd`(`D_Num`, `D_DateAch`, `D_Etat`, `F_Code`) VALUES ('2','2021-11-11','B','2');
```

```
INSERT INTO `dvd`(`D_Num`, `D_DateAch`, `D_Etat`, `F_Code`) VALUES ('3','2022-12-12','M','3');
```

```
INSERT INTO `parametre`(`Abo_Cout`, `Tva`) VALUES ('10','20')
```

```
INSERT INTO `emprunt`(`Num_Emp`, `Date_Emp`, `C_Numero`) VALUES ('1','2021-08-14','1');
```

```
INSERT INTO `emprunt`(`Num_Emp`, `Date_Emp`, `C_Numero`) VALUES ('2','2022-12-01','2');
```

```
INSERT INTO `emprunt`(`Num_Emp`, `Date_Emp`, `C_Numero`) VALUES ('3','2021-10-21','3');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('1','Drame');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('2','Science-fiction');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('3','Thriller');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('4','Fantastique');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('5','Horreur');
```

```
INSERT INTO `genre`(`G_Num`, `G_Int`) VALUES ('6','Animation');
```

```
INSERT INTO `rattacher`(`F_Code`, `G_Num`) VALUES ('1','1');
```

```
INSERT INTO `rattacher`(`F_Code`, `G_Num`) VALUES ('2','2');
```

```
INSERT INTO `rattacher`(`F_Code`, `G_Num`) VALUES ('3','6');
```

```
INSERT INTO `emprunter`(`D_Num`, `Num_Emp`, `Date_Ret`) VALUES ('1','1','2021-08-24');
```

```
INSERT INTO `emprunter`(`D_Num`, `Num_Emp`, `Date_Ret`) VALUES ('2','2','2022-12-11');
```

```
INSERT INTO `emprunter`(`D_Num`, `Num_Emp`, `Date_Ret`) VALUES ('3','3','2021-10-31');
```

- 14) Quel test proposez-vous pour tester l'Intégrité référentielle ?

```
INSERT INTO `dvd`(`D_Num`, `D_DateAch`, `D_Etat`, `F_Code`) VALUES ('4','2022-12-12','M','4'). Ne fonctionne pas car le film n°4 n'est pas renseigné.
```

- 15) Créer une requête qui affiche les prêts avec : le numéro du prêt, le code client, la date d'emprunt. Trier le résultat par numéro de prêt. Vérifier le résultat.

```
SELECT * FROM `emprunt` ORDER BY `Num_Emp`;
```

- 16) Créer une requête qui affiche pour chaque dvd emprunté et rapporté : le code type du client, le tarif journalier. Trier le résultat par prêt (à afficher). Vérifier le résultat.

```
SELECT type_client.`T_Code`, `Prixj`, client.`C_numero`, dvd.`D_Num`, emprunt.`Num_Emp`, emprunt.`Date_Emp`, emprunter.`Date_Ret` FROM `type_client` INNER JOIN client ON type_client.`T_Code` = client.`T_Code` INNER JOIN emprunt ON client.`C_numero` = emprunt.`C_Numero` INNER JOIN emprunter ON emprunt.`Num_Emp` = emprunter.`Num_Emp`;
```

```
unter.Num_Emp INNER JOIN dvd ON emprunter.D_Num = dvd.D_Num ORDER BY
emprunter.Num_Emp;
```

- 17) Créer une requête qui affiche pour chaque dvd emprunté et rapporté : la durée du prêt, son tarif journalier et son prix (durée*tarif journalier). Trier le résultat par prêt (à afficher). Vérifier le résultat.

```
SELECT dvd.D_Num, emprunt.Date_Emp, emprunter.Date_Ret, DATEDIFF(emprunter.
Date_Ret,emprunt.Date_Emp) AS nb_jours, type_client.Prixj, DATEDIFF(emprunter.Date
_Ret,emprunt.Date_Emp)*type_client.Prixj FROM `dvd` INNER JOIN emprunter ON dvd
.D_Num = emprunter.D_Num INNER JOIN emprunt ON emprunter.Num_Emp = emprunt
.Num_Emp INNER JOIN client ON emprunt.C_Numero = client.C_Numero INNER JOIN
type_client ON client.T_Code = type_client.T_Code ORDER BY emprunter.Num_Emp;
```

- 18) Créer une requête qui affiche pour chaque prêt son coût HT (durée de location de chaque DVD*tarif journalier). Vérifier son résultat.

```
SELECT emprunt.Num_Emp, emprunt.Date_Emp, emprunter.Date_Ret, DATEDIFF(
emprunter.Date_Ret,emprunt.Date_Emp) AS nb_jours, type_client.Prixj, DATEDIFF(
emprunter.Date_Ret,emprunt.Date_Emp)*type_client.Prixj FROM emprunt INNER J
OIN emprunter ON emprunt.Num_Emp = emprunter.Num_Emp INNER JOIN dvd O
N emprunter.D_Num = dvd.D_Num INNER JOIN client ON emprunt.C_Numero = cl
ient.C_Numero INNER JOIN type_client ON client.T_Code = type_client.T_Code;
```

- 19) Créer une fonction qui permet de calculer le coût TTC d'un prêt. Utiliser la fonction pour afficher le cout de chaque prêt et vérifier que vos tests sont cohérents avec les résultats de la requête du 14.

```
SELECT emprunt.Num_Emp, type_client.Prixj, DATEDIFF(emprunter.Date_Ret,emprunt.Date_Emp)
AS nb_jours, DATEDIFF(emprunter.Date_Ret,emprunt.Date_Emp)*type_client.Prixj AS prix_HT, par
ametre.Tva, DATEDIFF(emprunter.Date_Ret,emprunt.Date_Emp)*type_client.Prixj*(1+parametre.Tva
/100) AS prix_TTC FROM emprunt INNER JOIN emprunter ON emprunt.Num_Emp = emprunter.Nu
m_Emp INNER JOIN dvd ON emprunter.D_Num = dvd.D_Num INNER JOIN client ON emprunt.C_
Numero = client.C_Numero INNER JOIN type_client ON client.T_Code = type_client.T_Code, parametre;
```

- 20) Créer une vue qui affiche la liste des DVDs disponibles.

```
CREATE VIEW affichage_DVD_dispo AS SELECT * FROM dvd WHERE dvd.D_
Num NOT IN (SELECT dvd.D_Num FROM dvd INNER JOIN emprunter ON dvd.D
_Num = emprunter.D_Num);
```

- 21) Autoriser un nouvel utilisateur Local nommé client à voir cette liste (son mot de passe est XZclientZX, il sera crypté). Indiquez ici votre démarche. **Fait.**

GRANT SELECT ON dvddispo TO 'client'@'localhost';

Aide : <https://mariadb.com/kb/en/create-user/>

- 22) Connectez-vous en tant que client et vérifier son accès à cette vue. Coller ici la requête qu'il exécute pour voir la liste.

```
SELECT * FROM `affichage_dvd_dispo`
```

TP BD n°2 suite – Limovideo

fonctions

Une fonction retourne un résultat d'un certain type. Pour cela, elle peut avoir besoin qu'on lui fournisse des paramètres lors de l'appel : là aussi, il faut respecter leur type.

Elle peut aussi avoir besoin d'informations qui ont une valeur fixe (comme un taux de tva) qui dans ce cas sont stockées dans une table Paramètres.

Lors de la création et de l'utilisation des fonctions, demandez-vous toujours qu'elles sont les données à fournir lors de l'appel pour que la fonction puisse fournir le résultat demandé.

Un chapitre de cours sur les fonctions vous est fourni.

Conseil : avant de créer la fonction, tester d'abord si la requête qui la compose retourne le bon résultat pour des valeurs de paramètres que vous choisirez.

☞ Tester les fonctions suivantes et indiquer leur rôle:

NOW() → permet de retourner la date et l'heure du système

CURDATE() → renvoie une date basée sur une lecture de l'horloge de l'heure du jour

☞ Utiliser le script de la BD fourni, pour recréer une nouvelle BD limoTP2suite, de manière à ce que vous ayez tous la même BD pour créer ce TP.

Fait.

☞ Créer une fonction nbClientsType() qui affiche le nombre de clients d'un type de clients.

CREATE FUNCTION nbClientsType('T_Code' int(11))

RETURNS int

DETERMINISTIC

RETURN (

SELECT 'T_Code', COUNT('C_Numero') FROM 'client'

GROUP BY 'T_Code');

☞ Utiliser cette fonction pour obtenir le nombre de clients de type 1 (particuliers).

SELECT nbClientsType(1);

☞ Ecrire une requête qui utilise cette fonction pour afficher, pour chaque type existant, son nombre de clients (afficher le codetype et l'intitulé).

SELECT DISTINCT 'T_Code', nbClientsType('T_Code') FROM client;

☞ Ecrire la requête qui affiche la liste des dvds loués en août 2024 (afficher la date d'emprunt, le numéro du prêt et le numéro du dvd). Coller ici le résultat.

SELECT 'P_DateDeb', pret.'P_num', 'D_Num' FROM 'pret' INNER JOIN detailpret ON pret.P_num = detailpret.P_num WHERE 'P_DateDeb' LIKE '2024-08-%%';

P_DateDeb	P_num	D_Num
2024-08-11	110	1
2024-08-12	111	2

☞ Créer une fonction nbDvdsLouesMois() qui affiche le nombre de dvds loués pendant un mois donné (on fournit un entier pour le mois, et un autre pour l'année concernée).

CREATE FUNCTION nbDvdsLouesMois(mois int(2), annee int(4)) **RETURNS int DETERMINISTIC** **RETURN**(**SELECT COUNT**(D_Num`) **FROM** `pret` **INNER JOIN** detailpret **ON** pret.P\_num = detailpret.P\_num **WHERE** year(P\_DateDeb`)=annee **AND** month(P_DateDeb`)=mois);

☞ Coller ici la requête qui utilise la fonction créée pour afficher le nombre de dvds loués en août 2024 et vérifier si le résultat est cohérent par rapport à la liste des dvds loués obtenue précédemment.

SELECT nbDvdsLouesMois(08, 2024);

Le résultat est cohérent par rapport à la liste des dvds loués obtenue précédemment.

☞ Ecrire la requête qui affiche la liste des dvds loués dans le mois en cours (afficher la date d'emprunt, le numéro du prêt et le numéro du dvd, trier la liste par ordre chronologique). Coller ici le résultat.

SELECT `P\_DateDeb`, pret.`P\_num`, `D\_Num` **FROM** pret **INNER JOIN** detailpret **ON** pret.P_num = detailpret.P_num **WHERE** month(P_DateDeb`=month(now()) **AND** year(P\_DateDeb`=year(now()) **ORDER BY** `P\_DateDeb`;

☞ Créer une fonction nbDvdsLouesMoisC() qui affiche le nombre de dvds loués dans le mois en cours (le mois s'adapte suivant la date à laquelle on appelle la fonction).

CREATE FUNCTION nbDvdsLouesMoisC()
RETURNS int
DETERMINISTIC
RETURN(
SELECT COUNT(D_Num`) **FROM** pret
INNER JOIN detailpret
ON pret.P_num = detailpret.P_num
WHERE month(P_DateDeb`=month(now()) **AND** year(P\_DateDeb`=year(now())
ORDER BY `P\_DateDeb`);

☞ Coller ici la requête qui teste cette fonction et vérifier si le résultat est cohérent par rapport à la liste des dvds loués dans le mois en cours obtenue précédemment

SELECT nbDvdsLouesMoisC();

Le résultat est cohérent par rapport à la liste des dvds loués dans le mois en cours obtenue précédemment.

☞ Créer une fonction nbDvdsLouesAuj() qui affiche le nombre de dvds loués aujourd'hui.

```
CREATE FUNCTION nbDvdsLouesMoisAuj()  
RETURNS int  
DETERMINISTIC  
RETURN  
SELECT COUNT(D_Num) FROM pret  
INNER JOIN detailpret  
ON pret.P_num = detailpret.P_num  
WHERE month(P_DateDeb)=month(now()) AND year(P_DateDeb)=year(now())  
AND day(P_DateDeb)=day(now())  
ORDER BY P_DateDeb);
```

☞ Insérer les prêts suivants dans la BD en adaptant la date d'emprunt à la date de réalisation de votre TP.

```
INSERT INTO `pret` (`P_num`, `P_DateDeb`, `C_Numero`) VALUES  
(151, '2024-10-03', 2),  
(152, '2024-10-03', 4);
```

```
INSERT INTO `detailpret` (`D_Num`, `P_num`, `Date_ret`) VALUES  
(3, 151, 2024-10-04),  
(4, 152, 2024-10-04);
```

☞ Ecrire la requête qui affiche la liste des dvds loués aujourd'hui (afficher la date d'emprunt, le numéro du prêt et le numéro du dvd). Coller ici le résultat.

```
SELECT `P_DateDeb`, pret.`P_num`, `D_Num` FROM pret  
INNER JOIN detailpret  
ON pret.P_num = detailpret.P_num  
WHERE month(P_DateDeb)=month(now()) AND year(P_DateDeb)=year(now())  
AND day(P_DateDeb)=day(now())  
ORDER BY P_DateDeb;
```

☞ Coller ici la requête qui utilise la fonction créée pour afficher le nombre de dvds loués aujourd'hui et vérifier si le résultat est cohérent par rapport à la liste obtenue précédemment.

SELECT nbDvdsLouesMoisAuj();

Le résultat est cohérent par rapport à la liste obtenue précédemment.

Pour les plus avancés :

☞ Créer une fonction maxDvdLouesCli() qui affiche le plus grand nombre de dvds loués par un client.

☞ Utiliser cette fonction pour afficher le (ou les) clients(s) ayant réalisé ce plus grand nombre de dvd loués (afficher son numéro, son nom et le nb de dvd loués). Vérifier le résultat.

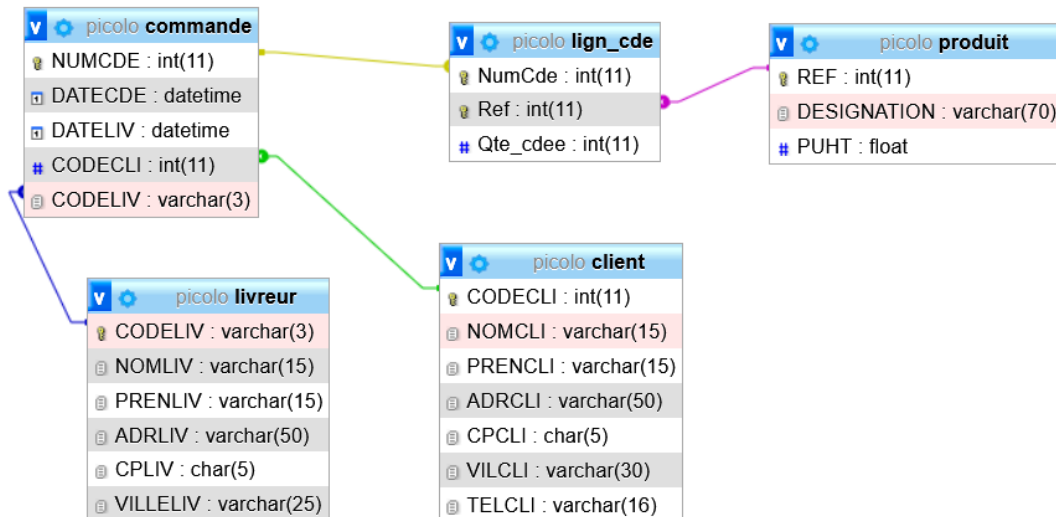
☞ Créer une fonction maxDvdsLouesMois() qui affiche le plus grand nombre de dvds loués en une journée dans le mois en cours.

☞ Utiliser cette fonction pour afficher la (ou les) journée(s) du mois en cours où Limovideo a réalisé ce plus grand nombre de dvd loués.

☞ Ajouter un prêt de votre choix pour l'une des journées affichées et ré-exécuter la fonction pour constater le changement de résultat.

TP n°3 Picolo suite

Rappel du schéma de la BD réalisée au TP n°1 (à vérifier car les liens devaient être créés à la question 5 du TP n°1). Si les liens ne sont pas faits dans votre BD, vous devez les créer à l'aide de requêtes.



1. Faites-en sorte que la date de livraison d'une commande ne puisse pas être avant la date de la commande.

ALTER TABLE commande **ADD CONSTRAINT** dateliv_datecde **CHECK** (DATELIV>DATECDE);

2. Ecrire les requêtes effectuées pour vérifier le bon fonctionnement de cette contrainte.

```
INSERT INTO `commande`(`NUMCDE`, `DATECDE`, `DATELIV`, `CODECLI`, `CODELIV`) VALUES ('11','2024-10-04','2024-10-03','2','L3')
```

La contrainte fonctionne comme prévu.

3. Créer une fonction qui permettra à partir d'un puht de calculer des prix TTC (le taux de TVA est fixé à 20%) et qui pourra être utilisé dans les requêtes, vues ...

CREATE FUNCTION calculPrixTTC(puht float) **RETURNS** float **DETERMINISTIC** **RETURN** (**SELECT** ('PUHT'*1.2) as prixTTC);

SELECT calculPrixTTC(4) = 4,8. Cela fonctionne.

4. Créer une vue qui permet de voir le total TTC des commandes du mois en cours (utiliser la fonction ttc).

Pour que la vue affiche une valeur il faut insérer une nouvelle commande et une nouvelle ligne commande.

CREATE VIEW totalTTCcommandes (prixTTCcommandes) **AS** **SELECT** calculPrixTTC(puht) **FROM** produit **INNER JOIN** lign_cde **ON** produit.REF = lign_cde.Ref **INNER JOIN** commande **ON** lign_cde.NumCde = commande.NUMCDE **WHERE** month('DATECDE')=month(now());

5. A l'aide de l'interface de PHPMyAdmin, créer un utilisateur Jdupont (mot de passe à noter ici : azerty12345) sans aucun droit sur des bases de données. Coller ici l'ordre SQL généré :

```
CREATE USER 'Jdupont'@'%' IDENTIFIED VIA mysql_native_password USING '***';GRANT USAGE ON *.* TO 'Jdupont'@'%' REQUIRE NONE WITH MAX_QUERIES_PER_HOUR 0 MAX_CONNECTIONS_PER_HOUR 0 MAX_UPDATES_PER_HOUR 0 MAX_USER_CONNECTIONS 0;
```

6. A l'aide d'une requête SQL (voir cours fourni) , autoriser le comptable connexion JDupont à voir cette vue.

```
GRANT SELECT ON lemaître_picolo.totalttccommandes TO 'Jdupont'@'%';
```

7. Ajouter une commande avec des lignes de commande pour le mois en cours. Tester la vue en vous connectant en tant que Jdupont.

```
INSERT INTO `commande`(`NUMCDE`, `DATECDE`, `DATELIV`, `CODECLI`, `CODELIV`) VALUES ('12','2024-10-01','2024-10-03','3','L2')
INSERT INTO `lign_cde`(`NumCde`, `Ref`, `Qte_cdee`) VALUES ('12','1277','25');
```

Test réussi.

8. Créer une fonction qui donne le chiffre d'affaires (ttc) réalisé avec un **produit donc la référence est passée en paramètre**. Avec une requête SQL, autoriser JDupont à l'utiliser (la syntaxe Mysql est GRANT EXECUTE ON FUNCTION nom_fonction TO user).

```
CREATE FUNCTION chiffreAffaireTTC (RF int)
RETURNS float
DETERMINISTIC
RETURN(
    SELECT SUM((produit.PUHT*1.2*Qte_cdee)) as Chiffre_daffaires FROM lign_cde
    INNER JOIN produit
    ON lign_cde.Ref = produit.REF
    WHERE produit.REF=RF
)
GRANT EXECUTE ON FUNCTION chiffreAffaireTTC TO 'Jdupont';
```

Test avec Jdupont : SELECT chiffreAffaireTTC(1250) Cela fonctionne.

9. Tester en affichant le chiffre d'affaires de chaque produit en étant connecté en root. Pour les plus avancés, faites en sorte que Null soit remplacé par 0 pour les produits jamais commandés.

SELECT chiffreAffaireTTC(1250) Fait.

10. Créer une procédure stockée (un cours vous est fourni) qui supprime les commandes d'un mois fourni en paramètre.

```
CREATE PROCEDURE effaceCommandeMois(IN p_mois int) BEGIN DELETE FROM lign_cde WHERE lign_cde.NumCde IN (SELECT NUMCDE FROM commande WHERE month(DATECDE)=p_mois); DELETE FROM commande WHERE month(DATECDE)=p_mois; END;
```

11. Sauvegarder votre BD et ses données et tester avec un mois pour lequel vous avez des commandes.

L'exportation est FAIT. call effaceCommandeMois(6) **Fait.**

12. Ajouter un champ *etat* (20 caractères) à la table *lign_cde*.

```
ALTER TABLE `lign_cde` ADD `etat` VARCHAR(20) NOT NULL ;
```

13. Ajouter un champ supplémentaire dans la table *produit* : la quantité en stock (*Qte_st*). Faire en sorte que tous vos produits aient une quantité en stock de 10.

```
ALTER TABLE produit ADD Qte_st int DEFAULT 10 ;
```

Pour les plus avancés :

14. Créer un trigger (le cours vous est fourni) qui permet, lors de l'enregistrement du détail d'une commande de vérifier que la quantité en stock des produits est suffisante (si une quantité en stock n'est pas suffisante alors Etat est affecté de 'en attente', sinon Etat est affecté de 'disponible').

```
DELIMITER |
```

```
CREATE TRIGGER verifie_QteSt BEFORE INSERT ON lign_cde FOR EACH ROW BEGIN  
DECLARE quantite int; SELECT Qte_St INTO quantite FROM produit WHERE NEW.ref =  
ref; IF (quantite < NEW.Qte_cdee) THEN SET NEW.etat=("En attente"); ELSE SET  
NEW.etat=("Disponible"); END IF; END;
```

15. Tester.

```
INSERT INTO `lign_cde`(`NumCde`, `Ref`, `Qte_cdee`, `etat`) VALUES ('12','1250','11','');
```

Résultat dans la base de donnée : En attente

```
INSERT INTO `lign_cde`(`NumCde`, `Ref`, `Qte_cdee`, `etat`) VALUES ('12','1284','5','');
```

Résultat dans la base de donnée : Disponible

NumCde	Ref	Qte_cdee	etat
12	1250	11	En attente
12	1277	25	
12	1284	5	Disponible

TP n°4 – Athlète

Vous disposez d'une base de données Athlete contenant une table *athlete* :

Athlete (num, nom, prenom, dateNaiss, cat)

num : clé primaire désigne le numéro de l'athlète

dateNaiss : désigne l'année de naissance de l'athlète

cat : désigne la catégorie de l'athlète, cet attribut peut prendre les valeurs suivantes :

- 1 si l'athlète a moins de 10 ans ou 10 ans
- 2 si l'athlète a plus de 10 ans et au maximum 18 ans
- 3 si l'athlète a plus de 18 ans

Rappel :

- Le rôle d'une fonction est de retourner un résultat (la plupart du temps résultat d'un calcul)
- Une procédure stockée est un script SQL qui permet d'effectuer une suite de commandes SQL. Elle peut éventuellement être programmée pour s'exécuter automatiquement suivant un planning donné.
- Un trigger se déclenche toujours automatiquement en liaison avec une mise à jour sur une table à laquelle il est attaché. A la création du trigger, on précise à quelle table il est lié et sur quelle commande SQL il se déclenche (INSERT, UPDATE, DELETE).
- Les fonctions, procédures stockées et triggers sont **stockés sur le serveur** de bases de données et sont **précompilés**, ce qui rend leur utilisation plus rapide.

TRAVAIL A FAIRE :

1. Créer la base de données nommée VotreNom_TP4athlete sous MariaDB.
Fait.
2. Créer la table *athlete* avec le script fourni.
Fait.
3. Écrire la fonction `age()` qui calcule l'âge de l'athlète à partir de sa date de naissance : vous utiliserez la fonction `TIMESTAMPDIFF`. Tester cette fonction pour tous les athlètes existants. Vérifier les résultats.
**CREATE FUNCTION age(dateNaiss` date) RETURNS int DETERMINISTIC
RETURN (SELECT TIMESTAMPDIFF(YEAR,dateNaiss,NOW()) as age);**
4. Ecrire la procédure stockée `majTouteCat()` qui va nous permettant de mettre à jour l'attribut *cat* de tous les athlètes déjà stockés en BD en fonction de leur âge.
**CREATE PROCEDURE majTouteCat() BEGIN UPDATE athlete SET cat = 1
WHERE age(dateNaiss`)<=10; UPDATE athlete SET cat = 2 WHERE
age(dateNaiss`)>10 AND age( dateNaiss`)<=18; UPDATE athlete SET cat = 3
WHERE age(dateNaiss`)>18; END;**
5. Exécuter la procédure, et vérifier le résultat pour tous les athlètes en affichant leur âge et en vérifiant si la catégorie générée par la procédure stockée est correcte.
**CALL majTouteCat();
SELECT *, age(dateNaiss`) FROM athlete;**

6. Pour les ajouts d'athlètes à venir : écrire un trigger qui permettra de renseigner automatiquement la catégorie. Indiquer les tests effectués pour valider votre travail (tester une insertion multiple : un seul INSERT qui ajoute plusieurs athlètes comme dans le script).

```
DELIMITER |
CREATE TRIGGER ajout_athlete
BEFORE INSERT
ON athlete
FOR EACH ROW
BEGIN
if(age(NEW.`dateNaiss`)<=10)
THEN
SET NEW.cat=1;
ELSE IF(age(NEW.`dateNaiss`)>18)
THEN
SET NEW.cat=3;
ELSE
SET NEW.cat=2;
END IF;
END IF;
END |
DELIMITER ;
```

```
INSERT INTO `athlete`(`nom`, `prenom`, `dateNaiss`, `cat`)
VALUES ('Durand','Serge','1955-02-17',NULL), ('Deschamps','Didier','1968-10-15',NULL);
```

7. Écrire un trigger permettant lors de la correction de la date de naissance d'un athlète de modifier automatiquement sa catégorie.

```
DELIMITER |
CREATE TRIGGER modif_Cat
BEFORE UPDATE
ON athlete
FOR EACH ROW
BEGIN
if(age(NEW.`dateNaiss`)<=10)
THEN
SET NEW.cat=1;
ELSE IF(age(NEW.`dateNaiss`)>18)
THEN
SET NEW.cat=3;
ELSE
SET NEW.cat=2;
END IF;
END IF;
END |
DELIMITER ;
```

```
UPDATE `athlete` SET `dateNaiss` = '2007-02-17' WHERE `athlete`.`num` = 6;
```

Evolution

La gestion des catégories en fonction de l'âge n'est pas optimale : le code est relativement lourd et pas du tout dynamique...et il n'y a que 3 catégories à gérer. Si une catégorie est ajoutée ou modifiée, cela impacte plusieurs codes qui seront à modifier.

Travail à faire :

1°) Faire une rétro-conception, pour proposer un Mcd qui gère les catégories et la tranche d'âge associée à chacune.

Fait.

2°) Gérer une BD AthleteV2 et remplissez la base de données.

Fait.

3°) Créer à nouveau la fonction age().

CREATE FUNCTION age(`dateNaiss` date) RETURNS int DETERMINISTIC RETURN (SELECT
TIMESTAMPDIFF(YEAR,dateNaiss,NOW()) as age);

4°) Créer une fonction cat() qui détermine la catégorie d'un athlète grâce à l'âge calculé par la fonction et au contenu de la table Categorie.

CREATE FUNCTION cat(age int)
RETURNS int
DETERMINISTIC
RETURN (
 SELECT categorie.`idCat` **FROM** categorie
 WHERE age>=ageMin **AND** age<=ageMax
)

5°) Créer la procédure majTouteCat() qui utilise cette fonction cat().

DELIMITER |
CREATE PROCEDURE majTouteCat ()
BEGIN UPDATE athlete
SET `idCat`= cat(age(`dateNaiss`));
END|

6°) Lancer la procédure et vérifier le résultat.

call majTouteCat() Résultat cohérent.

7°) Créer ensuite les triggers qui utiliseront eux aussi cette fonction cat(). Tester chaque trigger.

Trigger pour l'insertion :

DELIMITER |
CREATE TRIGGER insert_cat_athlete
BEFORE INSERT
ON athlete
FOR EACH ROW

```
BEGIN  
SET NEW.`idCat` = cat(age(NEW.`dateNaiss`));  
END|  
DELIMITER ;
```

Test qui le vérifie :

```
INSERT INTO `athlete`(`num`, `nom`, `prenom`, `dateNaiss`) VALUES  
('8','Dujardin','Jean','1972/07/19')
```

La catégorie a bien été rempli toute seule.

Trigger pour l'update :

```
DELIMITER |  
CREATE TRIGGER update_cat_athlete  
BEFORE UPDATE  
ON athlete  
FOR EACH ROW  
BEGIN  
SET NEW.`idCat` = cat(age(NEW.`dateNaiss`));  
END|  
DELIMITER ;
```

Test qui le vérifie :

```
UPDATE `athlete` SET `dateNaiss` = '2015-07-19' WHERE  
`athlete`.`num` = 8;
```

La catégorie est passé de 3 à 1.

TP n°5 Les transactions MariaDB (ou MySQL)

I. Le concept

A. Introduction

Les transactions sont une fonctionnalité absolument indispensable, permettant de sécuriser une application utilisant une base de données. Sans transactions, certaines opérations risqueraient d'être à moitié réalisées, et la moindre interruption pourrait avoir des conséquences énormes.

Une transaction est un ensemble de requêtes qui sont exécutées en un seul bloc. Ainsi, si une des requêtes du bloc échoue, on peut décider d'annuler tout le bloc de requêtes, cela afin de préserver l'intégrité des données de la base.

Remarque : les tables InnoDB supportent les transactions (pas les tables MyISAM).

B. Utilité des transactions

Imaginez que monsieur Durand fasse un virement de 300 euros à monsieur Dupont via sa banque en ligne. Il valide le virement : l'application de sa banque commence à le traiter mais soudain, une panne de courant provoque l'arrêt des serveurs de la banque.

Deux jours plus tard, monsieur Durand reçoit un appel de monsieur Dupont, très énervé, qui lui demande pourquoi le paiement convenu n'a toujours pas été fait. Intrigué, Monsieur Durand va vérifier son compte, et constate qu'il a bien été débité de 300 euros.

Que s'est-il donc passé ?

Normalement, le traitement d'un virement est plutôt simple, deux étapes suffisent :

1. étape 1 : on retire le montant du virement du compte du donneur d'ordre ;
2. étape 2 : on ajoute le montant du virement au compte du bénéficiaire.

Seulement voilà, la panne de courant qui a éteint les serveurs est survenue pile entre l'étape 1 et l'étape 2. Du coup, le compte de monsieur Durand a été débité, mais le compte de monsieur Dupont n'a jamais été crédité.

La banque de monsieur Durand n'utilisait pas les transactions. Si c'était le cas, la seconde requête du traitement n'ayant jamais été exécutée, la première requête n'aurait jamais été validée.

C. Transactions et cybersécurité : l'atomicité

Les transactions permettent d'assurer un concept important des bases de données relationnelles : l'**atomicité**. L'atomicité d'une transaction signifie que tout changement effectué doit être accompli jusqu'au bout. En cas d'interruption, par exemple une perte de connexion au milieu de l'opération, le changement doit être annulé et la base de données doit revenir automatiquement à son état antérieur (état au début de la transaction).

Ce principe permet d'éviter qu'une transaction soit partiellement terminée, à cause d'une panne ou d'un plantage. Dans le cas contraire, il est impossible de savoir à quel niveau d'avancée le processus a été interrompu. D'importantes complications peuvent s'en suivre.

Les transactions permettent ainsi de préserver l'intégrité des données de la base.

D. Comment se déroule une transaction ?

On démarre une transaction.

1. On exécute les requêtes désirées une à une.
2. Si une des requêtes échoue, on annule toutes les requêtes, et on termine la transaction.
3. Par contre, si à la fin des requêtes, tout s'est bien passé, on valide tous les changements, et on termine la transaction.
4. Si le traitement est interrompu (entre deux requêtes par exemple), les changements ne sont jamais validés, et les données de la base restent les mêmes qu'avant la transaction.

E. Valider/annuler les changements

Les commandes SQL pour valider (commiter) ou annuler des transactions sont relativement faciles à retenir :

COMMIT; -- pour valider une transaction (ou un bloc de requêtes)

ROLLBACK; -- pour annuler une transaction (ou un bloc de requêtes)

F. Comportement par défaut

Avec tous les TPs que vous avez réalisés, vous l'avez compris, par défaut MySQL et MariaDB ne travaillent pas avec les transactions. Chaque requête effectuée est directement **commitée** (validée). On ne peut pas revenir en arrière. On peut donc en fait considérer que chaque requête constitue une transaction, qui est automatiquement commitée. Par défaut, MySQL et MariaDb sont donc en mode "**autocommit**". Pour quitter ce mode, il suffit de lancer la requête suivante :

SET autocommit=0;

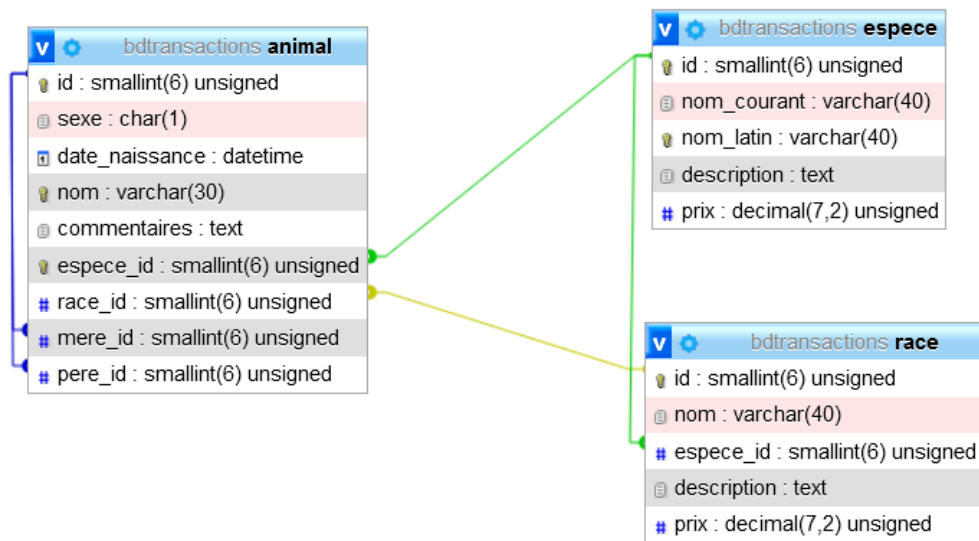
Cette commande ne peut être utilisée que dans une console de commandes (pas dans PHPMysqladmin).

Une fois que vous n'êtes plus en **autocommit**, chaque modification de donnée devra être commitée pour prendre effet. Tant que vos modifications ne sont pas validées, vous pouvez à tout moment les annuler (faire un rollback).

II. Mise en place d'un environnement de test

☞ créer la base de données VotreNom_BDtransactions sous MariaDB avec le script fourni grâce à PHPMysqladmin. Voici son schéma dans le concepteur :

Fait.



A. Questions sur la base de données :

Que signifie le lien de animal.id vers animal.mere_id ?

Pour chaque animal une mère est associé.

Que trouvera-t-on dans le champ animal.mere_id ?

Un identifiant qui permet de connaître la mère.

☞ A l'aide de la fonctionnalité Retro_conception, demander à Jmerise de créer le MCD correspondant à cette base de données. Vous nommerez les associations réflexives (qui relient l'entité animal à elle même) pour indiquer la nature (sémantique) de ce lien (voir MCD exemple fourni ci-après).

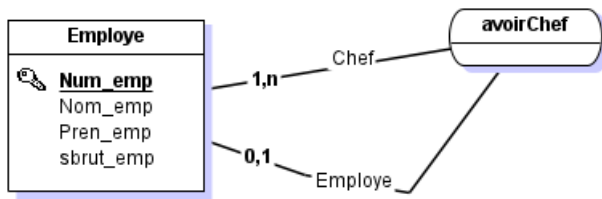
Fait.

☞ Jmerise a collé les deux branches des 2 associations réflexives et affiche **Plusieurs Liens**. A l'aide de l'outil double-cliquer sur l'une des branches marquées par **Plusieurs Liens** et dans la fenêtre qui apparaît :

- indiquer que vous souhaitez un ou 2 points de cassure (cela permettra aux 2 branches de se dissocier) **Fait.**

- indiquer également un nom et un code comme l'exemple ci-dessous le montre (attention à bien respecter les cardinalités). **Fait.**

Exemple de MCD avec une association réflexive qui montre qu'un employé à un chef qui est aussi stocké dans la table employé :



Respect des cardinalités :

Le nom Employee a été donné au lien qui porte la cardinalité 0,1 car un Employee a au maximum un chef et il peut en avoir aucun (s'il est le big boss par exemple)

Le nom chef a été donné à l'autre branche qui porte la cardinalité 1,n pour indiquer qu'un chef a au moins un employé sous sa responsabilité et au maximum n.

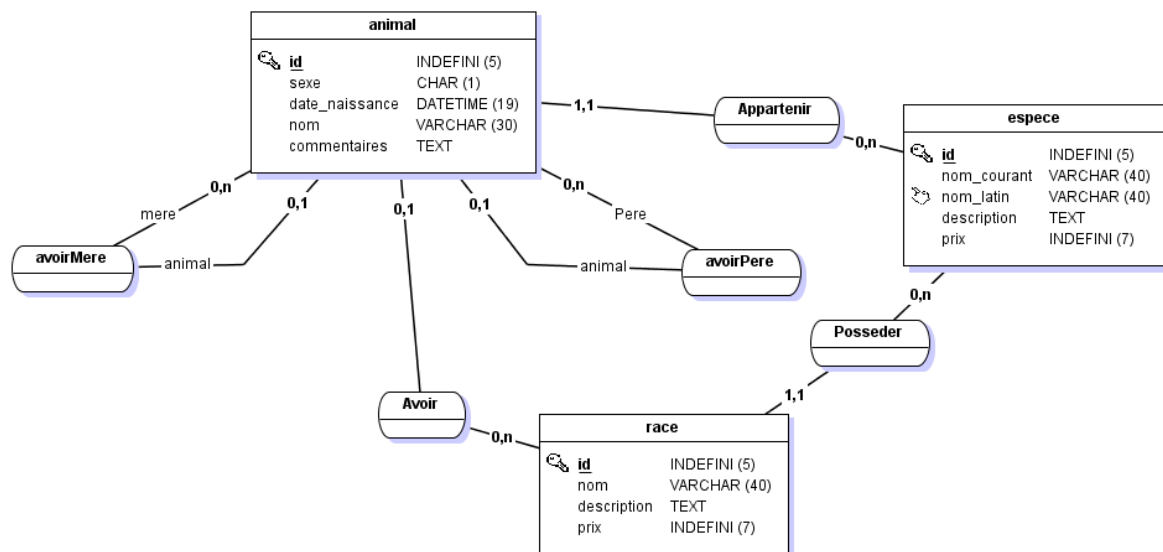
☞ Vérifier si le nom donné au lien correspond bien à la cardinalité.

Fait.

☞ Double-cliquer sur l'autre branche pour lui donner aussi un nom et un code.

Fait.

☞ Faites valider votre MCD et collez-le ici :



☞ Pour la suite, on va utiliser la console MySQL (car dans l'interface graphique, les requêtes sont systématiquement validées) : pour cela, déplacez-vous dans le dossier qui contient mysql.exe (chemin : C:\wamp64\bin\mariaDB\mariadb.x.xx\bin). Ouvrir l'invite de commande DOS en restant à cet endroit.

Fait.

☞ Puis entrer la commande `mysql.exe --user=root` qui permet d'exécuter des requêtes SQL en ligne de commande en étant connecté en tant que root

Fait.

☞ Puis indiquer à mariaDb qu'on utilise la BD VotreNom_BDtransactions grâce à la commande : `use Votre_nom_Bdtransactions;` **Fait.**

N.B. Pensez au ; à la fin des instructions.

Par la suite je parlerais de la BD comme étant BDTransactions.

☞ Ensuite continuer avec la commande `SET autocommit=0;`

Fait.

Cette commande indique à mariaDB de ne pas commiter (valider) automatiquement chaque bloc d'instructions, mais d'attendre un COMMIT pour cela.

III. Exemples de transactions

A. 1er exemple : annuler un bloc d'instructions SQL avec ROLLBACK

☞ Vérifier que vous n'avez pas d'animal de l'espèce avec id=5, grâce à la requête : `SELECT * FROM Animal WHERE espece_id = 5;` (vous pouvez faire un copier-coller)

Fait.

☞ Puis tester ce bloc d'instructions (avec un copier-coller du bloc) : **Fait.**

```
-- Insertion de 2 rats bruns
```

```
INSERT INTO Animal (nom, espece_id, date_naissance, sexe) VALUES ('Moustache', 5, '2023-11-13 17:45:00', 'M');
```

```
INSERT INTO Animal (nom, espece_id, date_naissance, sexe) VALUES ('Ratatouille', 5, '2023-11-13 17:48:00', 'F');
```

```
-- Les rats bruns coûtent maintenant 15 euros au lieu de 10 euros
```

```
UPDATE Espece SET prix = 15 WHERE id = 5;
```

```
-- Affichage des modifications effectuées
```

```
SELECT * FROM Animal WHERE espece_id = 5;
```

```
SELECT * FROM Espece WHERE id = 5;
```

☞ Examiner le résultat : **Fait.**

```
MariaDB [MJ_BDTransactions]> SELECT * FROM Animal WHERE espece_id = 5;
```

id	sexe	date_naissance	nom	commentaires	espece_id	race_id	mere_id	pere_id
63	M	2023-11-13 17:45:00	Moustache	NULL	5	NULL	NULL	NULL
64	F	2023-11-13 17:48:00	Ratatouille	NULL	5	NULL	NULL	NULL

```
2 rows in set (0.000 sec)
```

```
MariaDB [MJ_BDTransactions]> SELECT * FROM Espece WHERE id = 5;
```

id	nom_courant	nom_latin	description	prix
5	Rat brun	Rattus norvegicus	Petite bestiole avec de longues moustaches et une longue queue sans poils	15.00

```
1 row in set (0.001 sec)
```

Les INSERTs ont ajouté 2 animaux de l'espèce n°5, et un UPDATE a modifié le prix des animaux de l'espèce n°5.

☞ Maintenant réaliser la commande `ROLLBACK;`

Fait.

☞ Demander à revoir les animaux de l'espèce avec id=5 et les caractéristiques de cette espèce.

Fait.

```

MariaDB [BDTransactions]> ROLLBACK;
Query OK, 0 rows affected (0.006 sec)

MariaDB [BDTransactions]> SELECT * FROM Espece WHERE id =5;
+-----+-----+-----+-----+
| id | nom_courant | nom_latin | description | prix |
+-----+-----+-----+-----+
| 5 | Rat brun | Rattus norvegicus | Petite bestiole avec de longues moustaches et une longue queue sans poils | 10.00 |
+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [BDTransactions]> SELECT * FROM Animal WHERE espece_id = 5;
Empty set (0.000 sec)

```

Résultat : Vous pouvez constater que le ROLLBACK a annulé les instructions de la transaction (bloc d'instructions SQL), il n'y a plus d'animaux de l'espèce 5 et le prix est revenu à 10 :

B. 2ème exemple : Interruption de la transaction.

☞ Exécutez à nouveau les 2 requêtes **INSERT** et la requête **UPDATE**.

Fait.

☞ Déconnectez-vous du client MySQL en tapant exit.

Fait.

Question: sans regarder dans la table, pouvez-vous dire quel est le prix de l'espèce Rat brun Désormais ?

Le prix de l'espèce Rat brun est désormais de 10 euros.

☞ Vérifiez : Reconnectez-vous avec `mysql.exe --user=root` puis use `Bdtransactions`; et vérifiez vos données : les rats bruns valent 10 euros, et Moustache et Ratatouille n'existent pas. Les changements **n'ont pas été commités** (aucun commit effectué) !

Fait.

☞ Vérifiez si vous êtes en mode autocommit ou pas avec la commande `SELECT @@autocommit`;

Résultat à copier:

```

MariaDB [lemaitre_bdtransactions]> SELECT @@autocommit;
+-----+
| @@autocommit |
+-----+
| 1 |
+-----+
1 row in set (0.000 sec)

```

Explication :

Le mode autocommit est de nouveau activé ! Le `SET autocommit = 0`; n'est valable que pour la session courante. Or, en ouvrant une nouvelle connexion, vous avez créé une nouvelle session automatiquement en mode autocommit !

☞ Désactivez donc à nouveau ce mode.

Fait.

C. 3ème exemple : validation et annulation.

☞ Exécutez la séquence de requêtes suivante : **Fait.**

-- Insertion de 2 rats bruns

```
INSERT INTO Animal (nom, espece_id, date_naissance, sexe) VALUES ('Moustache', 5, '2023-11-13 17:45:00', 'M');
INSERT INTO Animal (nom, espece_id, date_naissance, sexe) VALUES ('Ratatouille', 5, '2023-11-13 17:48:00', 'F');
COMMIT;
-- Les rats bruns coûtent maintenant 20 euros au lieu de 10 euros
UPDATE Espece SET prix = 20 WHERE id = 5;
ROLLBACK;
```

☞ Indiquer ci-dessous le résultat obtenu et expliquer:

Les 2 insertions ont bien eu lieu. Par contre les rats brun ont toujours un prix à 10 euros car on a pas effectué de commit après.

D. 4ème exemple : visibilité des changements non-commités.

☞ Exécutez la requête suivante : **Fait.**

```
UPDATE Animal SET commentaires = 'Queue coupée' WHERE nom = 'Ratatouille' AND espece_id = 5;
```

☞ Ensuite, tout en laissant ce client MySQL ouvert, ouvrez-en un deuxième. Connectez-vous en root à votre base de données.

Fait.

☞ Sélectionnez les rats bruns avec la requête suivante : **Fait.**

```
SELECT id, sexe, nom, commentaires, espece_id, race_id
FROM Animal
WHERE espece_id = 5;
```

Vous constatez que le champ commentaires de Ratatouille est toujours vide. Les changements non-commités ne sont donc pas visibles à l'extérieur de la transaction qui les a réalisés. Une autre session n'a pas accès à ces changements (normal car ils n'ont pas été commités).

☞ Annulez la modification de Ratatouille dans la première session avec un ROLLBACK. Vous pouvez fermer la seconde session.

Fait.

IV. Démarrer explicitement une transaction

En désactivant le mode autocommit, en réalité, on démarre une transaction. Et chaque fois que l'on fait un rollback ou un commit (cela met fin à la transaction), une nouvelle transaction est créée automatiquement, et ce tant que la session est ouverte.

Il est également possible de démarrer explicitement une transaction avec START TRANSACTION, auquel cas on peut laisser le mode autocommit activé, et décider au cas par cas des requêtes qui doivent être faites dans une transaction.

Avec MySQL, il est également possible de démarrer une transaction avec BEGIN ou BEGIN WORK mais il est conseillé d'utiliser START TRANSACTION, car c'est la commande SQL standard.

Une fois la transaction ouverte, les requêtes devront être validées par un COMMIT pour prendre effet. Attention **un COMMIT ou un ROLLBACK met fin automatiquement à la**

transaction, donc les commandes qui suivront seront à nouveau committées automatiquement si une nouvelle transaction n'est pas ouverte.

☞ Repassez en mode autocommit. **Fait.**
Vérifiez. **Fait.**

A. Exemples de transactions en mode autocommit

```
-- Insertion d'un nouveau rat brun ☒
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Bianca', 5, '2023-11-13 02:25:00', 'F');

-- Insertion d'un nouveau rat brun, plus vieux ☒
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Momy', 5, '2022-02-01 02:25:00', 'F');

-- Ouverture d'une transaction
START TRANSACTION;
-- La nouvelle rate est la mère de Ratatouille et Moustache ☐
UPDATE Animal
SET mere_id = LAST_INSERT_ID()
WHERE espece_id = 5
AND nom IN ('Ratatouille', 'Moustache');
-- On annule les requêtes de la transaction, ce qui termine celle-ci
ROLLBACK;

-- La nouvelle rate est la mère de BIANCA ☒
UPDATE Animal
SET mere_id = LAST_INSERT_ID()
WHERE espece_id = 5
AND nom = 'Bianca';
-- Nouvelle transaction
START TRANSACTION;
-- Suppression de Moustache ☒
DELETE FROM Animal
WHERE espece_id = 5
AND nom = 'Moustache';
-- On valide les requêtes de la transaction, ce qui termine celle-ci
COMMIT;
```

☞ Indiquez par une case cochée ☒ les requêtes que vous pensez validées ou par une case non cochée ☐ les requêtes que vous pensez annulées. **Fait.**
Vérifiez. **Fait et les résultats obtenus sont ceux prévus.**

Cet exemple aurait pu également être testé en saisissant les requêtes dans PHPMyadmin qui est en mode autocommit !

B. Jalon de transaction

Lorsque l'on travaille dans une transaction, et que l'on constate que certaines requêtes posent problème, on n'a pas toujours envie de faire un rollback depuis le début de la transaction, annulant toutes les requêtes alors qu'une partie aurait pu être validée.

Il n'est pas possible de démarrer une transaction à l'intérieur d'une transaction. Par contre, on peut poser des **jalous de transaction**. Il s'agit de **points de repère**, qui permettent d'annuler toutes les requêtes exécutées depuis ce jalon, et non toutes les requêtes de la transaction.

Syntaxe

Trois nouvelles commandes suffisent pour pouvoir utiliser pleinement les jalons :

SAVEPOINT nom_jalon; -- Crée un jalon avec comme nom "nom_jalon"

ROLLBACK [WORK] TO [SAVEPOINT] nom_jalon; -- Annule les requêtes exécutées depuis le jalon "nom_jalon", WORK et SAVEPOINT sont facultatifs

RELEASE SAVEPOINT nom_jalon; -- Retire le jalon "nom_jalon" (sans annuler, ni valider les requêtes faites depuis)

Exemple :

```
START TRANSACTION;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Popi', 5, '2023-03-11 12:45:00', 'M');
SAVEPOINT jalon1;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Momo', 5, '2022-12-12 05:23:00', 'M');
ROLLBACK TO SAVEPOINT jalon1;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Mimi', 5, '2023-11-12 22:03:00', 'F');
COMMIT;
```

On n'a utilisé qu'une transaction, validée à la fin, et pourtant la seconde insertion n'a pas été faite, puisqu'elle a été annulée grâce au jalon (ROLLBACK TO SAVEPOINT jalon1). Seuls Popi et Mimi existent.

```
mysql> SELECT * FROM Animal WHERE espece_id = 5;
+----+-----+-----+-----+
| id | sexe | date_naissance | nom |
+----+-----+-----+-----+
| 76 | F    | 2023-11-13 17:48:00 | Ratatouille |
| 77 | F    | 2023-11-13 02:25:00 | Bianca |
| 78 | F    | 2022-02-01 02:25:00 | Momy |
| 79 | M    | 2023-03-11 12:45:00 | Popi |
| 81 | F    | 2023-11-12 22:03:00 | Mimi |
+----+-----+-----+-----+
```

V. Validation implicite et commandes non-annulables

Pour terminer une transaction, il faut utiliser les commandes **COMMIT** ou **ROLLBACK**, selon que l'on veut valider les requêtes ou les annuler. Ça, c'est la manière classique et recommandée.

Mais il faut savoir qu'un certain nombre d'autres commandes auront aussi pour effet de clôturer une transaction. Et pas seulement la clôturer, mais également **valider toutes les requêtes** qui ont été faites dans cette transaction. Exactement comme si vous utilisiez **COMMIT**. Par ailleurs, ces commandes ne peuvent pas être annulées par un **ROLLBACK**.

A. Commandes DDL

Toutes les commandes de définition de données qui créent, modifient, suppriment des objets (BD, tables, vues etc...) valident implicitement les transactions et ne peuvent pas être annulées par un **ROLLBACK**.

Ces commandes appelées requêtes DDL, pour *Data Definition Langage*, sont :

1. **CREATE DATABASE, DROP DATABASE** : création et suppression de bases de données ;
2. **CREATE TABLE, ALTER TABLE, RENAME TABLE, DROP TABLE** : création, modification, suppression de tables ;
3. **CREATE INDEX, DROP INDEX** : création, modification, suppression d'index ;
4. Celles pour la création d'objets comme les procédures stockées, les vues...

De manière générale, tout ce qui influe sur la **structure de la base de données**, et non sur les données elles-mêmes.

B. Utilisateurs

La création, la modification et la suppression d'utilisateurs provoquent aussi une validation implicite.

C. Transactions et verrous

La commande START TRANSACTION provoque également une validation implicite si elle est exécutée à l'intérieur d'une transaction.

Le fait d'activer le mode autocommit (s'il n'était pas déjà activé) a le même effet.

La création et suppression de verrous de table clôturent aussi une transaction en la validant implicitement (nous les verrons plus tard).

VI. ACID

Quels sont les **critères** qu'un système utilisant les transactions doit respecter pour être **fiable** ?

Ces critères sont au nombre de quatre : **Atomicité**, **Cohérence**, **Isolation** et **Durabilité**.

A pour Atomicité

Atome signifie étymologiquement "**qui ne peut être divisé**".

Une transaction doit être atomique, c'est-à-dire qu'elle doit former une **entité complète et indivisible**.

Si l'on reprend l'exemple du virement bancaire, en utilisant les transactions, les deux étapes (débit du compte donneur d'ordre, crédit du compte bénéficiaire) ne peuvent exister indépendamment l'une de l'autre. Si l'une est exécutée, l'autre doit l'être également. Il s'agit d'un tout.

Peut-on dire que nos transactions sont atomiques ?

Oui. Si une transaction en cours est interrompue, aucune des requêtes exécutées ne sera validée. De même, en cas d'erreur, il suffit de faire un **ROLLBACK** pour annuler toute la transaction. Et, un **COMMIT** lui, peut valider l'intégralité de la transaction en une fois.

C pour cohérence

Les données doivent rester **cohérentes dans tous les cas** : que la transaction se termine sans problème, qu'une erreur survienne, ou que la transaction soit interrompue. Un virement dont seule l'étape de débit du donneur d'ordre est exécutée produit des données incohérentes (la disparition de 300 euros jamais arrivés chez le bénéficiaire). Avec une transaction, cette incohérence n'apparaît jamais. Tant que la totalité des étapes n'a pas été réalisée avec succès, les données restent dans leur état initial.

Nos transactions permettent-elles d'assurer la cohérence des données ?

Oui, les changements de données ne sont validés qu'une fois que toutes les étapes ont été exécutées. De l'extérieur de la transaction, le moment entre les deux étapes d'un virement n'est jamais visible.

I pour Isolation

Chaque transaction doit être isolée, donc **ne pas interagir avec une autre transaction**.

Nos transactions sont-elles isolées ?

Test

☞ Dans une première console en mode auto-commit désactivé, exécuter les requêtes suivantes (ne committez pas) pour modifier le *pere_id* du rat Bianca (si le rat N° 71 n'existe pas, indiquez le n° de Pipo): **Fait avec le n°62.**

```
START TRANSACTION; -- On ouvre une transaction
UPDATE Animal -- On modifie Bianca
SET pere_id = 71
WHERE espece_id = 5 AND nom = 'Bianca';
SELECT id, nom, commentaires, pere_id, mere_id
FROM Animal
WHERE espece_id = 5;
```

☞ Dans une 2ème console mysql en mode auto-commit désactivé, exécutez les requêtes suivantes, pour modifier les *commentaires* de Bianca (prenez bien tout le bloc d'un coup, vous suivrez plus facilement les explications qui suivent). **Fait.**

```
START TRANSACTION; -- On ouvre une transaction
SELECT id, nom, commentaires, pere_id, mere_id
FROM Animal
WHERE espece_id = 5;

UPDATE Animal -- On modifie pour Bianca
SET commentaires = 'Sympathique et souriante'
WHERE espece_id = 5 AND nom = 'Bianca';
SELECT id, nom, commentaires, pere_id, mere_id
FROM Animal
WHERE espece_id = 5;
```

Le résultat n'est pas du tout le même dans les deux sessions. En effet, dans la première, on a la confirmation que la requête **UPDATE** a été effectuée. Et le **SELECT** renvoie bien les données modifiées (*pere_id* n'est plus **NULL** pour Bianca).

```
mysql> SELECT id, nom, commentaires, pere_id, mere_id
-> FROM Animal
-> WHERE espece_id = 5;
```

id	nom	commentaires	pere_id	mere_id
68	Ratatouille	NULL	NULL	NULL
69	Bianca	NULL	71	70

Dans la seconde, la requête **UPDATE** ne fait rien ! La session semble bloquée : pas de message de confirmation après la requête **UPDATE**, et le second **SELECT** n'a pas été effectué.

```
mysql> SELECT id, nom, commentaires, pere_id, mere_id
-> FROM Animal
-> WHERE espece_id = 5;
```

id	nom	commentaires	pere_id	mere_id
68	Ratatouille	NULL	NULL	NULL
69	Bianca	NULL	NULL	70
70	Momy	NULL	NULL	NULL
71	Popi	NULL	NULL	NULL
73	Mimi	NULL	NULL	NULL

5 rows in set (0.00 sec)

```
mysql>
mysql> UPDATE Animal -- On modifie pour Bianca
-> SET commentaires = 'Sympathique et souriante'
-> WHERE espece_id = 5 AND nom = 'Bianca';
```

☞ Commitez maintenant les changements dans la première session (celle qui n'est pas bloquée). **Fait.**

Dans l'autre console, les modifications ayant été faites par la session 1, elles ont été prises en compte : *commentaires* vaut **'Sympathique et souriante'** et *pere_id* vaut 71 !

```
mysql> UPDATE Animal -- On modifie pour Bianca
-> SET commentaires = 'Sympathique et souriante'
-> WHERE espece_id = 5 AND nom = 'Bianca';
Query OK, 1 row affected (2 min 11.63 sec)
Enregistrements correspondants: 1 Modifiés: 1 Warnings: 0

mysql> SELECT id, nom, commentaires, pere_id, mere_id
-> FROM Animal
-> WHERE espece_id = 5;
```

id	nom	commentaires	pere_id	mere_id
68	Ratatouille	NULL	NULL	NULL
69	Bianca	Sympathique et souriante	71	70
70	Momy	NULL	NULL	NULL
71	Popi	NULL	NULL	NULL
73	Mimi	NULL	NULL	NULL

5 rows in set (0.00 sec)

Il est possible que votre seconde session indique ceci :

ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction . Cela signifie que la session est restée bloquée trop longtemps et que par conséquent la transaction a été automatiquement fermée (avec un rollback des requêtes effectuées). Dans ce cas, recommencez l'opération.

☞ commiter les changements faits par la deuxième session, et c'est terminé !

Fait.

Si vous ne committez pas, *commentaires* restera **NULL**. Par contre, *pere_id* vaudra toujours 71 puisque ce changement-là a été commité par la première session.

Conclusion

La deuxième session n'a pas interagi avec les changements faits par la première session, chaque transaction est bien **isolée**.

Et la première session qui bloque la seconde, ce n'est pas une interaction ça ?

Pas dans le cadre des critères ACID. Oui, la première session provoque un retard dans l'exécution des requêtes de la deuxième session, mais les critères de fiabilité que nous examinons ici concernent **les données impactées par les transactions**, et non le déroulement de celles-ci (qui importe peu finalement).

Ce blocage a pour effet d'empêcher la deuxième session d'écraser un changement fait par la première. Donc, ce blocage a bien pour effet l'isolation des transactions.

Verrous

Le blocage de la deuxième session vient en fait de ce que la première session, en faisant sa requête **UPDATE**, a **automatiquement posé un verrou** sur la ligne concernant Bianca , empêchant toute modification tant que la transaction était en cours. Les verrous feront l'objet d'un prochain chapitre.

Utilité

Vous n'allez pas vous amuser tous les jours à ouvrir deux sessions MySQL. Par contre, pour une application pouvant être utilisée par plusieurs personnes en même temps (qui toutes travaillent sur la même base de données), il est impératif que ce critère soit respecté.

Prenons l'exemple simple d'un jeu par navigateur : de nombreux joueurs peuvent être connectés en même temps, et effectuer des opérations différentes. Si les transactions ne sont pas isolées, une partie des actions des joueurs risquerait de se voir annulées.

On isole donc les transactions grâce aux verrous (qui sont ici automatiquement posés mais ce n'est pas toujours le cas).

D pour Durabilité

Une fois la transaction terminée, les données résultant de cette transaction doivent être **stockées de manière durable**, et pouvoir être récupérées, en cas de crash du serveur par exemple.

Nos transactions modifient-elles les données de manière durable ?

Oui, une fois les changements commités, ils sont stockés définitivement (jusqu'à modification par une nouvelle transaction).

VII. Les transactions dans du code PHP

A. Exemple 1 :

On veut que les deux opérations UPDATE soient réalisées ou aucune.

```
<?php
require 'connexion.php';
$connexion=connect_bd();
$montant = 100;
// On commence la transaction
$connexion->beginTransaction()
// Retrait du Compte1
$stmt1 = $connexion->prepare('
    UPDATE compte
    SET solde = solde - :montant
    WHERE num = :nm
');
$cpte1 = '12342345677';
$stmt1->bindParam(':num', $cpte1);
$stmt1->bindParam(':montant', $montant, PDO::PARAM_INT);
$stmt1->execute();

// Credit du Compte2

$stmt2 = $connexion->prepare('
    UPDATE compte
    SET solde = solde + :montant
    WHERE num = :num
');
$cpte2 = '98787324898954895';
$stmt2->bindParam(':nom', $cpte2);
$stmt2->bindParam(':montant', $montant, PDO::PARAM_INT);
$stmt2->execute();
//on termine la transaction
$connexion->commit();
```

?>

Dans l'exemple ci-dessous, on peut aussi faire les prepare AVANT de démarrer la transaction et les execute() qui rendent effective les modifications DANS la transaction.

B. Application :

Vous utiliserez la notion de transaction, pour améliorer le code de la mission 2 GsbParam afin qu'une commande ne puisse pas être enregistrée sans les lignes de commande qu'elle contient.

Vous testerez par exemple en renommant temporairement la table *contenir* pour que l'instruction INSERT INTO contenir échoue.

L'application devra gérer l'échec de l'enregistrement de la commande par un affichage adapté et vous devrez vérifier que la commande n'a pas été enregistrée en BD.

VIII. En résumé

Les transactions permettent de **grouper plusieurs requêtes**, lesquelles seront validées (**COMMIT**) ou annulées (**ROLLBACK**) toutes en même temps.

Tous les changements de données (insertion, suppression, modification) faits par les requêtes à l'intérieur d'une transaction sont **invisibles pour les autres sessions tant que la transaction n'est pas validée**.

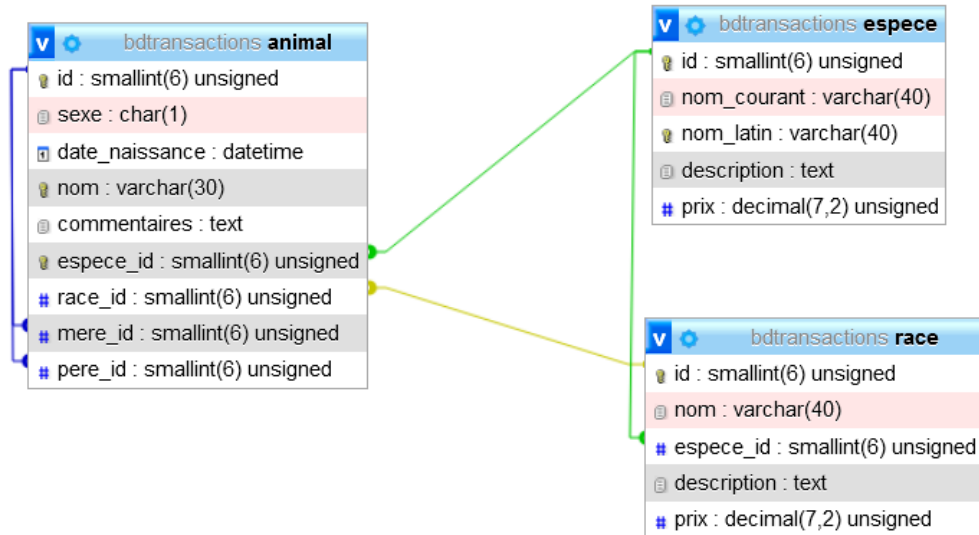
Les transactions permettent d'**exécuter un traitement nécessitant plusieurs requêtes en une seule fois**, ou de l'annuler complètement si une des requêtes pose problème ou si la transaction est interrompue.

Certaines commandes SQL provoquent une **validation implicite des transactions**, notamment toutes les commandes DDL, c'est-à-dire les commandes qui créent, modifient ou suppriment des objets dans la base de données (tables, index,...).

Les critères ACID sont les critères qu'un système appliquant les transactions doit respecter pour être fiable : **Atomicité, Cohérence, Isolation, Durabilité**.

TP n°5 requêtes BD transactions

La Base de données



Les données de la table animal (extrait) :

id	sexe	date_naissance	nom	commentaires	espece_id	race_id	mere_id	pere_id
1	M	2010-04-05 13:43:00	Rox	Mordille beaucoup	1	1	18	22
2	NULL	2010-03-24 02:23:00	Roucky	NULL	2	NULL	40	30
3	F	2010-09-13 15:02:00	Schtroumpfette	NULL	2	4	41	31
...								
17	F	2008-03-10 13:45:00	Welva	NULL	1	NULL	NULL	NULL
18	F	2007-04-24 12:59:00	Zira	NULL	1	1	NULL	NULL
19	F	2009-05-26 09:02:00	Java	NULL	1	2	NULL	NULL
...								
22	M	2007-04-24 12:42:00	Bouli	NULL	1	1	NULL	NULL
24	M	2007-04-12 05:23:00	Cartouche	NULL	1	NULL	NULL	NULL
25	M	2006-05-14 15:50:00	Zambo	NULL	1	1	NULL	NULL
26	M	2006-05-14 15:48:00	Samba	NULL	1	1	NULL	NULL
27	M	2008-03-10 13:40:00	Moka	NULL	1	NULL	NULL	NULL
28	M	2006-05-14 15:40:00	Pilou	NULL	1	1	NULL	NULL
29	M	2009-05-14 06:30:00	Fiero	NULL	2	3	NULL	NULL
30	M	2007-03-12 12:05:00	Zonko	NULL	2	5	NULL	NULL

I. Quelques requêtes

A. 'De tête'

En observant les lignes ci-dessus extraites de la table animal, indiquez moi quel est le nom du père de Rox ? **Bouli** Quel est le nom de sa mère ? **Zira**

B. Votre démarche

Pour répondre aux questions, vous avez d'abord chercher la valeur de `pere_id` dans la ligne où le nom de l'animal est Rox.

Ensuite vous avez chercher cette valeur (celle de `pere_id`) dans la colonne `id` de `animal` : vous avez fait une jointure sur le critère `pere_id` de la table d'origine = `id` d'une copie de la table `animal` (renommée par exemple `pere`) et ensuite vous avez chercher le nom de l'animal qui correspond.

Remarque on ne peut pas travailler en même temps sur 2 lignes différentes de la même table. C'est pour cette raison que je vous indique qu'il vous faut une copie de la table d'origine : vous savez le faire avec un **alias**, exemple FROM `animal` **pere**.

Table d'origine : animal

id	sexe	date_naissance	nom	commentaires	espece_id	race_id	mere_id	pere_id
1	M	2010-04-05 13:43:00	Rox	Mordille beaucoup	1	1	18	22
2	NULL	2010-03-24 02:23:00	Roucky	NULL	2	NULL	40	30
3	F	2010-09-13 15:02:00	Schtroumpfette	NULL	2	4	41	31

Copie nommée : pere

22	M	2007-04-24 12:42:00	Bouli	NULL	1	1	NULL	NULL
24	M	2007-04-12 05:23:00	Cartouche	NULL	1	NULL	NULL	NULL
25	M	2006-05-14 15:50:00	Zambo	NULL	1	1	NULL	NULL
26	M	2006-05-14 15:48:00	Samba	NULL	1	1	NULL	NULL
27	M	2008-03-10 13:40:00	Moka	NULL	1	NULL	NULL	NULL
28	M	2006-05-14 15:40:00	Pilou	NULL	1	1	NULL	NULL
29	M	2009-05-14 06:30:00	Fiero	NULL	2	3	NULL	NULL
30	M	2007-03-12 12:05:00	Zonko	NULL	2	5	NULL	NULL

C. La requête SQL (auto-jointure)

☞ Tester la requête suivante : **Fait**.

```
SELECT animal.nom, pere.nom AS 'pere'
FROM Animal
INNER JOIN animal pere
ON animal.pere_id=pere.id
WHERE animal.nom='ROX';
```

nom	pere
Rox	Bouli

On retrouve ci-dessus

- la sélection de la ligne pour laquelle `animal.nom='ROX'`
- la récupération de `id` de son pere grâce à `animal.id_pere`
- la jointure vers une copie nommée `pere` sur le critère `animal.pere_id=pere.id`
- la récupération du nom du père grâce à `pere.nom`

☞ Compléter la requête pour trouver le nom de la mère de Rox.

```

SELECT animal.nom, mere.nom AS 'Mere', pere.nom AS 'pere'
FROM Animal
INNER JOIN animal mere
ON animal.mere_id = mere.id
INNER JOIN animal pere
ON animal.pere_id=pere.id
WHERE animal.nom='ROX';

```

On obtient :

nom	Mere	pere
Rox	Zira	Bouli

☞ Combien y-a-il d'animaux dans la table ?

```

SELECT COUNT(id) AS 'Nombre d'animaux' FROM `animal`

```

Nombre d'animaux
63

☞ Réaliser la requête pour trouver le père et la mère de chaque animal. Avez-vous bien le père et la mère de CHAQUE animal ? Expliquez.

```

SELECT animal.nom, mere.nom AS 'Mere', pere.nom AS 'pere'
FROM Animal
INNER JOIN animal mere
ON animal.mere_id = mere.id
INNER JOIN animal pere
ON animal.pere_id=pere.id
GROUP BY animal.nom;

```

nom	Mere	pere
Bianca	Momy	Pipo
Bobo	Caroline	Pataude
Roucky	Milla	Zonko
Rox	Zira	Bouli
Schtroumpfette	Feta	Filou

Il y a pas tous les animaux car la plupart n'ont pas de mère ou de père.

☞ Ajouter une mère à Bobosse, et un père à Canaille.

```

UPDATE `animal` SET `mere_id` = '3' WHERE `animal`.`id` = 6;

```

```

UPDATE `animal` SET `pere_id` = '20' WHERE `animal`.`id` = 11;

```

☞ Corriger la requête pour trouver le père et la mère de chaque animal (la jointure ne doit pas éliminer des lignes de la tables animal).

```

SELECT animal.nom, mere.nom AS 'Mere', pere.nom AS 'pere'
FROM Animal
LEFT JOIN animal mere
ON animal.mere_id = mere.id
LEFT JOIN animal pere
ON animal.pere_id=pere.id
GROUP BY animal.nom;

```


Remarque : La jointure d'une table avec elle même est appelée **auto-jointure**.

D. Les contraintes

☞ Avez-vous veillé à ce que le père de Canaille soit un animal de sexe Masculin ? De la même espèce? Et né avant Canaille ...

Fait aux questions suivantes.

☞ Mettre en place les outils nécessaires pour mettre en place une de ces contraintes (pour les plus avancés , vous pouvez tout gérer).

```
DELIMITER |
CREATE TRIGGER verifPere
BEFORE INSERT
ON animal
FOR EACH ROW
BEGIN
    IF (SELECT sexe FROM animal WHERE id = NEW.pere_id) != 'M' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le père doit être de sexe masculin';
    END IF;
    IF (SELECT espece_id FROM animal WHERE id = NEW.pere_id) !=
NEW.espece_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'L'espece du père doit être similaire à celui du
fils';
    END IF;
    IF (SELECT date_naissance FROM animal WHERE id =
NEW.pere_id) >= NEW.date_naissance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le père doit être né avant le fils';
    END IF;
END |
DELIMITER ;
```

☞ Tester.

```
UPDATE `animal` SET `pere_id` = '37' WHERE `animal`.`id` = 4;
```

MySQL a répondu : 🐛

```
#1644 - Le père doit être de sexe masculin
```

```
UPDATE `animal` SET `pere_id` = '62' WHERE `animal`.`id` = 4;
```

MySQL a répondu : 🐛

```
#1644 - L'espece du père doit être
similaire à celui du fils
```

```
UPDATE `animal` SET `pere_id` = '32' WHERE `animal`.`id` = 46;
```

MySQL a répondu : 

#1644 - Le père doit être né avant le fils

TP n°6 Urubu

Ce TP doit donner lieu à un compte rendu, où vous collerez les requêtes réalisées et indiquerez les difficultés rencontrées et les solutions trouvées.

Concepts de cybersécurité abordés : les autorisations avec des rôles MySQL ainsi que assurer l'intégrité des données.

Contexte :

La Sté Urubu & Cie vous confie la base de données relationnelle qui permet d'enregistrer ces ventes et de les associer au vendeur (table ccial) ayant réalisé celles-ci. Voici le modèle relationnel souhaité :

ARTICLE (ref_art, lib_art, pu_art)

Clé primaire : ref_art

RELEVE (idReleve, date_vte, ref_art, no_vendeur, qte)

Clé primaire : idReleve

Clé étrangère ref_art qui fait ref à ref_art de article

Clé étrangère no_vendeur qui fait no_vendeur de vendeur

CCIAL (no_vendeur, nom, prenom, ville)

Clé primaire : no_vendeur

Etude et modification de la structure de la base de données

☞ Modifiez le script *creabase* pour que la clé primaire de RELEVE soit auto-incrémentée.

Fait.

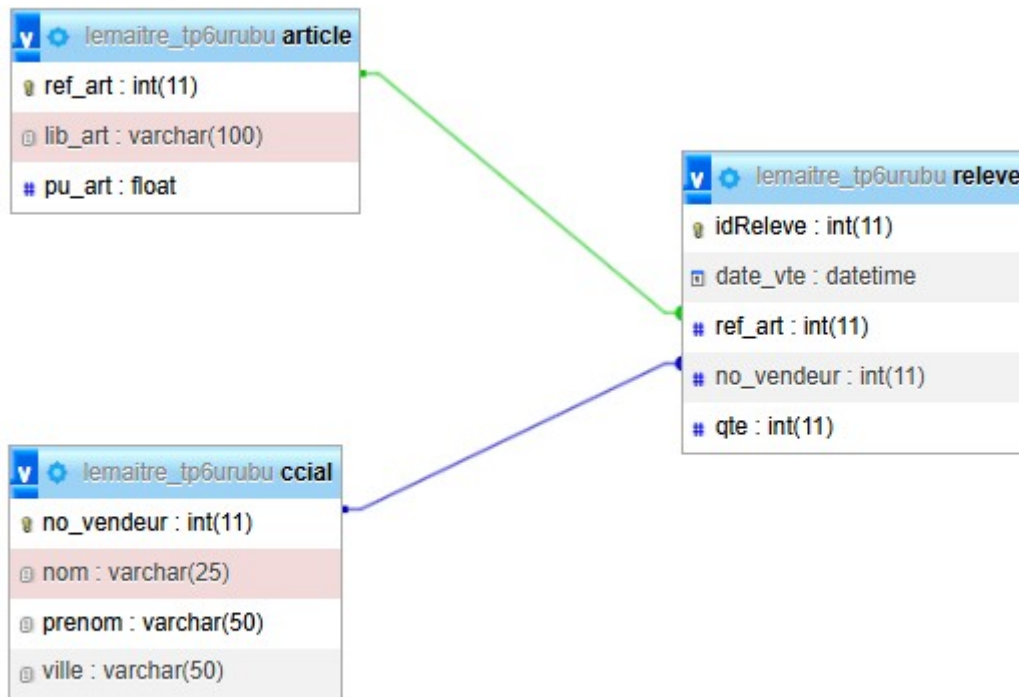
☞ Modifiez le script *creabase* : ajoutez les contraintes de clés étrangères

Fait.

☞ Créez la base de données votreNom_TP6Urubu sous MariaDB à l'aide du script modifié.

Fait.

☞ Coller ci-dessous le schéma de la base de données vue du concepteur.



Les données

☞ Peuplez les tables à l'aide du script *lesDonnees.sql*

Fait.

Les requêtes

☞ Ecrire en SQL les requêtes permettant de satisfaire les besoins suivants (vous supprimerez les doublons quand cela vous semblera approprié):

1. Afficher les références et libellés de tous les articles avec le nombre de ventes les plus vendus (nb ventes) en premier.

SELECT article.`ref\_art`, `lib\_art`, COUNT(releve.no_vendeur) as nbVentes FROM `article`

INNER JOIN releve

ON article.ref_art = releve.ref_art

GROUP BY article.ref_art

ORDER BY nbVentes DESC

ref_art	lib_art	nb ventes
1002	Logitech ERGO K860 Clavier Ergonomique Sans Fil	8
1041	Corsair Ventilateur de boîtier, Noir, 120 mm	5
1008	Phomemo M08F - Imprimante Portable, Imprimante Com...	5
1043	Corsair LL120 RGB, Ventilateur de Boîtier Dual Lig...	4
1040	Bewinner Carte Graphique de Jeu, Carte Graphique 1...	2
1046	HanCenDa Câble de chargement magnétique	2
1025	Toshiba 1TB Canvio Basics Portable External Hard D...	2
1005	Perixx PERIDUO-605, Combo Ergonomique, Clavier et ...	2
1034	Carte de Capture vidéo Mukaensi, HDMI	2
1038	ASUS Carte Graphique GT710-SL-2GD3-BRK-EVO	1
1015	Timetec 8 Go KIT (2x4Go) DDR3L 1333MHz PC3-10600	1
1047	TUGAU Tablette D'écriture LCD Adulte 15 Pouces Doo...	1
1042	Corsair SP120 Elite, Ventilateur pour boîtier de 1...	1

2. Même question que le 2, mais les articles non vendus apparaissent avec 0 vente (utiliser une jointure externe). Vous devez voir apparaître :

**SELECT article.`ref\_art`, `lib\_art`, COUNT(releve.no_vendeur) as nbVentes
FROM `article`**

LEFT JOIN releve

ON article.ref_art = releve.ref_art

GROUP BY article.ref_art

ORDER BY nbVentes DESC

1003	Perixx PERIDUO-406, Jeu de Clavier Filaire Compact...	0
1078	SYNOLOGY DS223j Boîtier NAS	0
1026	WD 2 TB Elements Portable External Hard Drive - US...	0
1013	Crucial Pro RAM 48GB Kit (2x24Go) DDR5 5600MHz (ou...	0
1051	Processeur AMD Ryzen 7 5800X Socket AM4 (3,8 Ghz)	0
1022	KOORUI Écran PC Gaming 27 Pouces, FHD (1920 x 1080...	0
1009	HP DeskJet 2720e Imprimante tout en un - Jet d'enc...	0
1080	Synology DS120j 1 Bay Desktop NAS Enclosure, Black	0
1032	Carte d'acquisition vidéo Newhope, Carte de Captur...	0
1019	Corsair Vengeance LPX CMK32GX4M2E3200C16 Module de...	0
1052	Intel Processeur Core i5-12400F pour PC de bureau ...	0
1023	Weighmax Portable Disque Dur Externe, 2To Ultra-Mi...	0

3. Afficher le montant total des ventes réalisées par Gates (c'est son nom).

```
SELECT `nom`, `prenom`, SUM(releve.qte*article.pu_art) as totalVentes FROM
`ccial`
INNER JOIN releve
ON ccial.no_vendeur = releve.no_vendeur
INNER JOIN article
ON releve.ref_art = article.ref_art
WHERE nom = 'GATES'
```

nom	prenom	total ventes
GATES	Bill	4237.72

4. Afficher les mois de 2023 pour lesquels le cumul des ventes est supérieur à 10000€.

```
SELECT MONTH(`date_vte`) as mois, SUM(releve.qte*article.pu_art) as
totalVentes FROM `releve`
INNER JOIN article
ON releve.ref_art = article.ref_art
GROUP BY mois
HAVING totalVentes > 10000
```

mois	total ventes
8	11605.20

5. Créer une vue nonCommandes2023 qui indique les Réf. et libellé des articles pour lesquels il n'y a pas eu de vente en 2023. Tester.

```
CREATE VIEW nonCommandes2023
AS SELECT `ref_art`, `lib_art`, `pu_art` FROM article
WHERE `ref_art` NOT IN (
  SELECT DISTINCT releve.ref_art
  FROM releve
  WHERE YEAR(releve.date_vte) = 2023
)
```

ref_art	lib_art	pu_art
1003	Perixx PERIDUO-406, Jeu de Clavier Filaire Compact...	29.99
1009	HP DeskJet 2720e Imprimante tout en un - Jet d'enc...	49.99
1010	Epson Imprimante Expression Home XP-2200, Multifon...	49.95
1011	Mémoire RAM 16Go Kit (2x8Go) DDR3 1600MHz	26
1013	Crucial Pro RAM 48GB Kit (2x24Go) DDR5 5600MHz (ou...	140
1019	Corsair Vengeance LPX CMK32GX4M2E3200C16 Module de...	84
1020	KOORUI Écran PC gaming 22 Pouces Full HD	84
1021	Dell SE2422HX 24" Full HD (1920x1080) Écran PC, 75...	110.89
1022	KOORUI Écran PC Gaming 27 Pouces, FHD (1920 x 1080...	179.99
1023	Weighmax Portable Disque Dur Externe, 2To Ultra-Mi...	45.99
1024	Boitier Disque Dur 2.5, ORICO 6Gbps USB C 3.1 Gen ...	12.99
1026	WD 2 TB Elements Portable External Hard Drive - US...	79.99
1032	Carte d'acquisition vidéo Newhope, Carte de Captur...	28.24
1045	Memoria SODIMM DDR4 3200 16GB C22 Team Elite	34.97
1049	Carte Mere GIGABYTE B550M DS3H	99.8
1051	Processeur AMD Ryzen 7 5800X Socket AM4 (3,8 Ghz)	219.13
1052	Intel Processeur Core i5-12400F pour PC de bureau ...	164.1
1065	JONSBO D41 Mesh Screen ATX Gehäuse, Tempered Glass...	132.77
1078	SYNOLOGY DS223j Boîtier NAS	204
1080	Synology DS120j 1 Bay Desktop NAS Enclosure, Black	109.78

6. Nous souhaitons autoriser tous les vendeurs à consulter cette vue. Pour cela vous allez devoir :

- créer un rôle d'application nommé Vendeurs (**un cours vous est fourni**).

CREATE ROLE Vendeurs;

- autoriser le rôle vendeurs à consulter la vue créée précédemment.

GRANT SELECT ON noncommandes2023 TO Vendeurs;

- créer ensuite un utilisateur Eve_DUPONT qui aura les droits du rôle vendeurs (noter sont mot de passe qui servira dans le TP suivant : azerty).

Fait.

- activer le rôle vendeurs pour Eve DUPONT

GRANT Vendeurs TO 'Eve_DUPONT';

SET DEFAULT ROLE Vendeurs FOR 'Eve_DUPONT';

Pour en savoir plus sur les rôles : <https://mariadb.com/kb/en/create-role/>

7. Indiquer comment vous testez l'autorisation donnée au rôle vendeurs.

Pour tester les autorisations données au rôle Vendeurs, il faut reproduire les actions autorisées ou interdites pour ce rôle avec l'utilisateur Eve_DUPONT.

Evolution de la base de données.

Gestion des zones géographiques :

Chaque commercial est affecté à une zone géographique qui est caractérisée par un numéro de zone, un libellé de zone et une superficie. Par exemple :

- Les vendeurs N°12, N°14, N°18 sont affectés à la zone 1 (nom : Centre, superficie 39151 km²)
- Les vendeurs N°13, N°15, N°16 à la zone 2... (nom : Bretagne, superficie 27208 km²)

☞ Indiquez votre réflexion et les ordres SQL nécessaires pour mettre à jour la BD (pour la syntaxe vous pouvez vous aider du script SQL fourni pour la création de la BD).

- Création de la table zone :

```
CREATE TABLE zone (
  num_zone INTEGER NOT NULL AUTO_INCREMENT,
  nom_zone VARCHAR(100),
  superficie INT NULL,
  PRIMARY KEY (num_zone)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
```

- Ajout d'un champ id_zone dans la table ccial :

```
ALTER TABLE `ccial` ADD `num_zone` INT NOT NULL;
```

- Ajout d'informations dans la table zone :

```
INSERT INTO `zone`(`nom_zone`, `superficie`) VALUES ('Centre',39151)
INSERT INTO `zone`(`nom_zone`, `superficie`) VALUES ('Bretagne',27208)
```

- Ajout de la contrainte de clé étrangère dans la table ccial qui fait référence à id_zone de la table zone :

```
ALTER TABLE ccial
ADD FOREIGN KEY (num_zone)
REFERENCES zone(num_zone);
```

☞ Vérifier votre travail : essayez d'attribuer la zone n°3 au vendeur n°11. Que se passe-t-il ? Expliquez pourquoi.

Cela ne fonctionne pas car la zone 3 n'existe pas dans la table zone.

☞ Vérifier votre travail en affichant tous les vendeurs et leur zone (num_zone et nom_zone). On doit voir apparaître aussi les vendeurs sans zone.

```
SELECT `no_vendeur`, `nom`, `prenom`, ccial.`num_zone`, zone.nom_zone FROM
`ccial`
LEFT JOIN zone
ON ccial.num_zone = zone.num_zone
```

Gestion des formations.

Pour motiver ses commerciaux, la société leur propose des formations. Une formation est caractérisée par un numéro, un intitulé, une date de début et une date de fin. A l'issue d'une formation, chaque commercial remplit un questionnaire de satisfaction et attribue une note (de 1 à 5) à cette formation. La société souhaite enregistrer les formations suivies par chaque commercial et les notes qu'il a attribuées à chacune. Cela lui permettra ensuite de voir quelles formations sont les plus appréciées.

☞ Indiquez les ordres SQL nécessaires pour mettre à jour la BD.

- Création de la table formation :

```
CREATE TABLE formation (
  num_for INTEGER NOT NULL AUTO_INCREMENT,
  lib_for VARCHAR(100),
  date_Deb DATETIME NULL,
  date_Fin DATETIME NULL,
  note INT NULL,
  PRIMARY KEY (num_for)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
```

- Ajout d'un champ num_for dans la table ccial :

```
ALTER TABLE `ccial` ADD `num_for` INT NULL;
```

- Ajout d'un champ note dans la table ccial :

```
ALTER TABLE `ccial` ADD `note` INT NULL;
```

- Ajout d'informations dans la table formation :

```
INSERT INTO `formation`(`lib_for`, `date_Deb`, `date_Fin`) VALUES ('Framework CodeIgniter','2024-12-01 09:00:00','2024-12-02 17:00:00')
INSERT INTO `formation`(`lib_for`, `date_Deb`, `date_Fin`) VALUES ('Framework Symfony','2025-01-05 09:30:00','2025-01-09 17:00:00')
```

- Ajout des contraintes de clé étrangère dans la table ccial qui font référence aux champs la table formation :

```
ALTER TABLE ccial
ADD FOREIGN KEY (`num_for`)
REFERENCES formation(num_for);
```

```
ALTER TABLE ccial
ADD FOREIGN KEY (`note`)
REFERENCES formation(note);
```

Vous pourrez créer :

- une formation intitulée 'Framework CodeIgniter' qui démarre le 1er décembre à 9h et finit le 2 à 17h.
- une formation intitulée 'Framework Symfony' qui démarre le 5 janvier à 9h30 et finit le 9 janvier à 17h.

☞ Comment la société pourra-t-elle faire un classement des formations (de celles ayant eu la meilleure moyenne à celle ayant eu la moins bonne moyenne) ? Votre réponse doit être précise.

Création d'une vue qui affiche chaque formation avec une note moyenne.

Pour obtenir les notes moyennes il faut calcul la somme des notes puis diviser par le nombre de notes pour chaque formation.

Puis faire un ORDER BY DESC des notes moyennes pour obtenir le classement des notes moyennes avec la formation qui correspond.

Vous pourrez créer des participations et des notes pour tester le classement. Que se passe-t-il si on insère une note incorrecte égale à 6, par exemple. Remédiez à cela.

```
ALTER TABLE formation  
ADD CONSTRAINT CK_note CHECK (note BETWEEN 1 and 5)
```

Procédures stockées.

☞ Ecrire les procédures stockées qui répondent aux besoins suivants (pensez à mettre des commentaires dans le code de vos procédures stockées) :

1. Procédure *MajPrix* qui permet d'augmenter les PUHT des articles à partir d'un taux fourni en paramètre.

```
DELIMITER |  
CREATE PROCEDURE MajPrix (IN taux float) -- taux d'augmentation en  
pourcentage  
BEGIN  
IF taux < 0 THEN -- Vérifie que le taux est positif  
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT = 'Le taux d'augmentation doit être positif';  
END IF;  
UPDATE article  
SET pu_art = pu_art * (1 + taux / 100);  
END|  
DELIMITER ;
```

CALL MajPrix(10) Fonctionne.

2. Procédure stockée *Arch_vtes* qui archive les ventes de l'année précédente :
une table *Arch_ventes* sera d'abord créée en dehors de la procédure stockée (elle contient la date de vente, la référence de l'article et la quantité vendue, inutile de créer les clés).
La procédure stockée, fera en sorte que les ventes de l'année 20XX soient enregistrées dans cette table *Arch_ventes* et soient ensuite supprimées de la table *RELEVE*.

Création de la table Arch_ventes :

```
CREATE TABLE Arch_ventes (
  dateVente DATETIME NOT NULL,
  ref_art INT NOT NULL,
  qte INT NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
```

Création de la procédure stockée Arch_vtes :

```
DELIMITER |
CREATE PROCEDURE Arch_vtes (IN annee INT)
BEGIN
  INSERT INTO arch_ventes (dateVente, ref_art, qte) -- Ajoute dans la table
arch_ventes les informations de l'année choisie
  SELECT date_vte, ref_art, qte
  FROM releve
  WHERE YEAR(date_vte) = annee;

  DELETE FROM releve -- Supprime ce qui a été archivé dans la table releve
  WHERE YEAR(date_vte) = annee;
END|
DELIMITER ;
```

CALL Arch_vtes(2023) Fonctionne.

Attention, exécuter cette procédure en ayant au préalable :

- Fait une sauvegarde (un export de votre BD et de ses données)
Fait.
- vérifier que la table RELEVÉ contient des ventes pour l'année précédente et l'année en cours.
Fait.

Pour les plus avancés : faites-en sorte que la table Arch_ventes20XX soit créée automatiquement par la procédure (son nom changera automatiquement chaque année). Tester.

Remarque : on ne peut pas utiliser une requête construite avec une variable (comme ici le nom de la table) dans une procédure stockée. Il faut procéder par étapes. Voir l'article

<https://copyprogramming.com/howto/mysql-dynamically-build-query-string-in-a-stored-procedure-based-on-logic>

qui vous donne un exemple pour un SELECT mais le principe est le même pour le CREATE TABLE et les autres requêtes à faire dans cette procédure stockée.

TP n°7 Sauvegarder une BD MySQL

Compétences cybersécurité abordées : assurer l'intégrité et la confidentialité des données.

I. Introduction

A. Cyberattaque et sauvegarde.

Face à la recrudescence des cyberattaques qui sont souvent à l'origine de pertes ou de vol de données, sauvegarder les données informatiques en entreprise est devenu une nécessité **même si les données qu'elle stocke n'ont rien de sensible**.

Si une attaque de type **ransomware** (demande de rançon en échange du retour des données) a lieu et que les données sont chiffrées par le logiciel pirate, seule la sauvegarde de données permettra de retrouver un fonctionnement normal. Il suffira pour cela de récupérer la dernière sauvegarde afin de restaurer les données.

B. Sans sauvegarde de ses données, une entreprise risque la faillite

Que cela soit dû à une attaque informatique ou à un sinistre (inondation...), les conséquences d'une perte de données peuvent être fatales. On ne parle pas seulement de la mise hors ligne du site web de l'entreprise, on parle de **dégâts engendrant des pertes financières directes et indirectes** :

- **perte des données commerciales** (base de données d'un site e-commerce) ou de **réservations** (hôtellerie, restauration, etc.) ;
- mise **hors service du matériel informatique** ;
- **impossibilité pour les salariés de travailler** pendant parfois plusieurs jours.

60% des PME mettent la clé sous la porte dans les 18 mois suivants une cyberattaque.

C. Qualité des sauvegardes ?

Une sauvegarde de données (aussi appelée **backup**), désigne une copie des fichiers stockés (sur un ordinateur ou un serveur).

Lors d'une sauvegarde, il est recommandé de **multiplier les supports contenant les données** pour avoir une base de secours en cas de pertes, pour une récupération rapide et sans difficulté du fonctionnement normal des services informatiques.

La manière la plus sécurisée est de stocker au minimum une sauvegarde sur un support physique déconnecté du réseau et stocké dans un lieu sécurisé contre les vols, les incendies etc..

Une sauvegarde doit être réalisée régulièrement pour être efficace et pour conserver les données les plus récentes possibles.

Il existe plusieurs supports pour réaliser des sauvegardes :

- la clé USB (capacité trop limitée pour les entreprises et risque de perte),
- le disque dur externe,

- les NAS (Network Attached Storage) permettent de connecter en réseau sur le même disque dur un ensemble de postes et d'automatiser les sauvegardes de ces derniers. Mais il reste connecté au réseau

La sauvegarde des données entreprises peut également être réalisée sur le cloud : Ici, l'entreprise ne gère pas elle-même les supports physiques ; ce sont des prestataires qui stockent les données en ligne sur des serveurs distants, sécurisés. On enlève donc certains désavantages des stockages sur disques : pas d'erreurs humaines (du côté de l'entreprise), pas de perte ni d'oubli de sauvegarder, et un stockage dans un lieu sécurisé.

Si une entreprise fait le choix du backup de données sur le cloud, il convient de faire un comparatif des solutions de sauvegarde existantes, mais surtout d'effectuer un chiffrement des données avant de les confier à un tiers.

Les outils publics et connus comme Google Drive, Dropbox, etc... **ne sont pas des solutions recommandées** pour des sauvegardes de données d'entreprise. Il est plutôt conseillé de s'orienter vers des organismes tiers spécialisés dans cette thématique : **Veritas Backup Exec, Spectrum Protect Suite, Veeam, Rubrik**, etc.

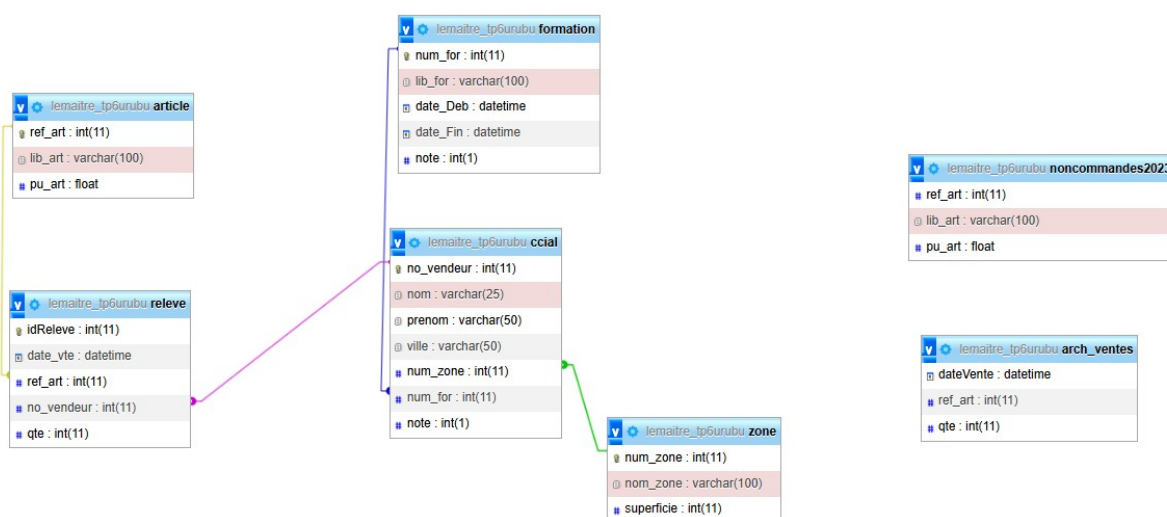
Dans ce TP, nous allons aborder la sauvegarde des bases de données MySQL (ou MariaDB).

II. La sauvegarde d'une base de données MySQL

A. En mode graphique

1. Duplication de base

☞ Sous PhpMyAdmin, positionné sur votre base de données **urubu**, faites une capture d'écran du concepteur montrant le schéma de votre base de données.



☞ Choisissez "Copier la base" dans l'onglet Opérations.

☞ Appelez la copie `svg_urubu` et exécutez la copie. Observez le résultat dans le concepteur pour comparer au schéma de la BD original capturé.

Fait.

Que constatez-vous ? **Le résultat est le même**

Où est stocké la copie de votre base de données par rapport à l'original ? **En dehors de mon dossier personnel.**

Ce lieu de stockage est-il satisfaisant ? pourquoi ? **Non car tout le monde peut la voir.**

Conclusion sur la copie de base de données pour sauvegarder ? **Pour sauvegarder la copie de la base de donnée il faut la mettre dans le repertoire de la personne qui s'en occupe pour pouvoir gerer les droits plus facilement sur cette base.**

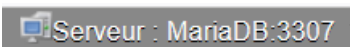
2. Export de votre base

Une base de données peut-être aussi être créée à partir d'un script SQL. Pour cela, il faut un script complet comprenant en plus de la création des tables, des fonctions, des déclencheurs ..., la création des clés étrangères et parfois, les ordres de création de la base et d'insertion des données. Vous allez donc générer le script SQL de votre base `urubu`.

☞ Sous PhpMyAdmin, positionné sur votre base de données `urubu`, choisissez une exportation personnalisée dans l'onglet **Exporter**.

Fait.

3. Export de plusieurs bases

☞ Sous PhpMyAdmin, positionné sur aucune base de données (donc au niveau serveur ) , choisissez une exportation personnalisée dans l'onglet **Exporter**. Vous constatez que vous pouvez sélectionner plusieurs bases de données à sauvegarder (par défaut le nom du script sera celui du serveur). **Fait.**

4. Sauvegarde partielle

Dans certains cas, il peut être nécessaire de sauvegarder une seule table plutôt que toute une base.

☞ Sous PhpMyAdmin, positionné sur votre base de données **urubu**, choisissez une exportation personnalisée dans l'onglet **Exporter**.

☞ Sélectionnez la table **releve**, l'affichage des résultats et des relations de clés étrangères. Le reste des options étant bien positionné par défaut, générez le script (il portera par défaut le nom de la table sauvegardée). **Fait.**

On peut aussi copier une table (en étant positionné sur cette table et en sélectionnant **Copier la table** vers dans **Opérations**).

Question :

Rappelez ici l'ordre SQL qui permettrait de faire une copie des enregistrements de la table **releve** vers une table existante nommée **svg_releve** ayant la même structure :

```
DELIMITER |
CREATE PROCEDURE Sauvegarde_releve ()
BEGIN
    INSERT INTO svg_releve (dateVente, ref_art, qte) -- Ajoute dans la table
    svg_releve les informations de l'année choisie
    SELECT date_vte, ref_art, qte
    FROM releve

END|
DELIMITER ;
```

B. En mode console

Pour sauvegarder des bases de données en mode console, il existe l'utilitaire **MySQLdump** qui permet d'exporter une base ou un groupe de bases vers un fichier texte, pour la sauvegarde ou le transfert entre 2 serveurs (pas nécessairement entre serveurs MySQL). L'export contiendra les requêtes SQL nécessaires pour créer la table et la remplir.

Un des avantages de **mysqldump** réside dans la possibilité de transférer les données sur un autre type de serveur car le fichier généré par **mysqldump** ne contient que des ordres SQL et d'effectuer des sauvegardes distantes, c'est à dire, sauvegarder une base (située sur un serveur) à partir d'un autre PC.

☞ Créer un dossier **testSauvegardesBD** dans le dossier documents sur C : (que je nommerais <C:/users/VotreNom/documents/>) **Fait**

☞ Avec l'explorateur de Windows, déplacez-vous dans le dossier qui contient **mysql.exe** (soit pour une BD créée sous Mysql : **C:\wamp64\bin\mysql\mysqlx.y.v\bin**, ou pour une BD MariaDB. **C:\wamp64\bin\mariadb\mariadb.x.y.v\bin**) **Fait.**

☞ Ouvrir l'invite de commande DOS en restant à cet endroit.

Fait.

Pour les sauvegardes qui vous sont demandées ci-après, vous noterez l'instruction et conserverez chaque script de sauvegarde.

1. Sauvegarder une base de données

☞ Générez l'ordre SQL sauvegarde de votre base **urubu** avec l'instruction suivante :

```
mysqldump -u root svg_urubu > C:/users/VotreNom/documents/testSauvegardsBD/svg_urubu.sql
```

Fait.

L'utilisateur doit bien sûr soit être administrateur du SGBDR, soit propriétaire de la BD, soit avoir reçu des droits d'effectuer des sauvegardes.

2. Autorisations pour sauvegarder une base de données

a) Un exemple

Exemple pour que Eve_DUPONT crée dans le TP n°6, sauvegarde la BD urubu :

```
GRANT USAGE ON urubu.* TO 'Eve_DUPONT'@'localhost';
GRANT SELECT, LOCK TABLES ON urubu.* TO 'Eve_DUPONT'@'localhost';

GRANT CREATE, INSERT, DROP ON mysql.backup_history_old TO
'Eve_DUPONT'@'localhost';

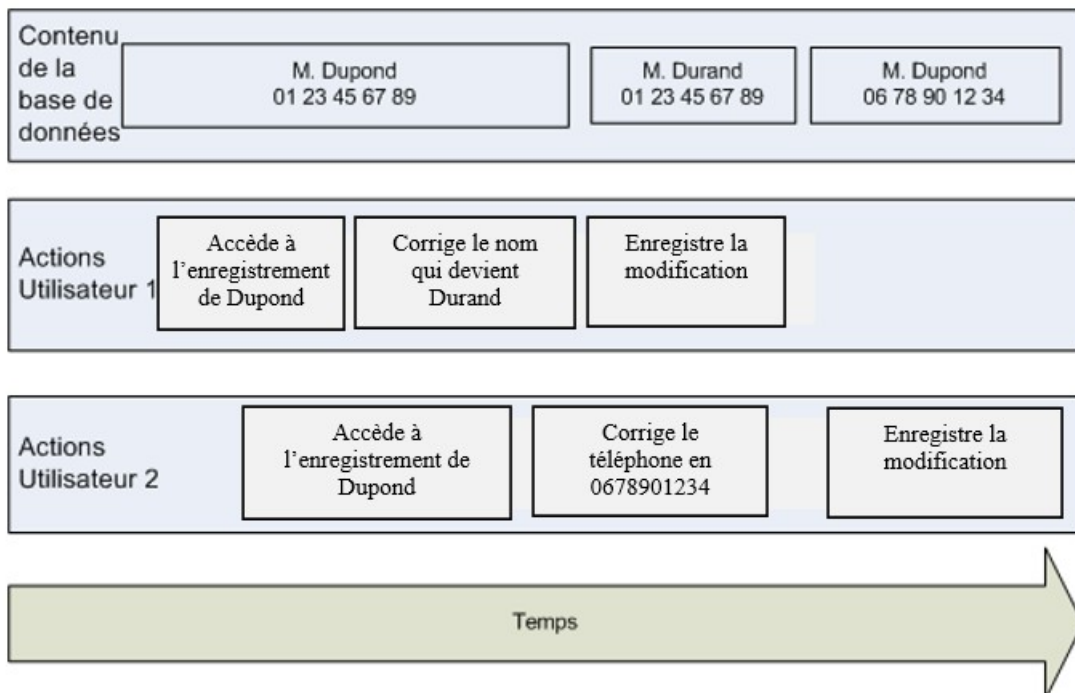
GRANT CREATE, INSERT, DROP, ALTER ON mysql.backup_history_new TO
'Eve_DUPONT'@'localhost';

GRANT SHOW VIEW ON urubu.* TO 'Eve_DUPONT'@'localhost';
```

PS les droits à ajouter peuvent varier (le GRANT SHOW VIEW a été ajouté pour la vue nonCommande2023). Donc bien lire les messages d'erreur affichés.

b) Les verrous de table et cohérence des données

Les SGBDs permettent à plusieurs utilisateurs de modifier des données d'un même enregistrement en même temps : dans ce cas, il y a un risque de mémoriser une incohérence. L'image ci-dessous présente ce cas avec la modification d'un enregistrement qui contient les coordonnées de M. Dupond.



Sur cette image, on voit qu'il y a un risque ici que seule la modification du dernier utilisateur soit conservée. Pour résoudre ce conflit MySQL utilise une technique de verrou.

c) Le principe du verrou

Le verrou permet de bloquer l'accès aux données verrouillées aux autres utilisateurs. Lorsqu'un utilisateur souhaite accéder à une donnée verrouillée, il sera mis dans une file d'attente, et aura accès aux données une fois que l'utilisateur qui a posé le verrou, l'aura levé.

Ces verrous sont principalement de 2 types :

- Ils peuvent empêcher tout accès aux données aux autres utilisateurs.
- Ils peuvent laisser uniquement un accès en lecture seule aux autres utilisateurs.

Le verrou peut être mis automatiquement par MySQL lors d'une sauvegarde pour empêcher l'accès en modification des données pendant la sauvegarde, ou la lecture d'un enregistrement pendant sa modification. Il peut aussi être mis manuellement par une instruction LOCK TABLE puis sera ensuite supprimé pour libérer l'accès pour des modifications.

3. Sqldump avec utilisateur et saisie du mot de passe

Pour autoriser un utilisateur qui a un mot de passe, on ajoute l'option -p pour la saisie du mot de passe, la commande devient :

```
mysqldump -u user -p databasename > C:/users/VotreNom/documents/testSauvegardesBD/svg_urubu2.sql
```

donc pour Eve_Dupont :

```
mysqldump -u Eve_DUPONT -p urubu > C:/users/VotreNom/documents/testSauvegardesBD/svg_urubu2.sql
```

4. Sauvegarder toutes les bases de données

Pour sauvegarder plusieurs bases de données, vous pouvez utiliser la syntaxe suivante :

```
mysqldump -u db_username -p --databases db_name1 db_name2 > backup_file.sql
```

Pour sauvegarder toutes les bases de données, vous pouvez utiliser la syntaxe suivante :

```
mysqldump -u db_username -p --all-databases > all_databases_backup_file.sql
```

☞ Faites un dump de toutes les bases de données : pensez à préciser le chemin vers **C:\wamp64\bin\mariadb\mariadb10.10.2\bin\all_databases_backup_file.sql**.

```
mysqldump -u root -p --all-databases > all_databases_backup_file.sql
```

☞ Vérifiez

Fait résultat cohérent.

5. Sqldump : les options

Il est possible de préciser une table en particulier à sauvegarder avec le paramètre **--tables nom_table** et aussi d'affecter une clause de restriction sur les lignes sélectionnées avec le paramètre **--where critère_de_restriction**.

☞ Faites un dump de la table releve uniquement.

```
mysqldump -u root -p lemaitre_tp6urubu --tables releve > releve_backup_file.sql
```

☞ Vérifiez

Fait résultat cohérent.

☞ Complétez votre ordre mysqldump en sauvegardant uniquement les lignes de la table releve pour lesquelles ref_art=1041.

```
mysqldump -u root -p lemaitre_tp6urubu --tables releve --where ref_art=1041 > releve1041_backup_file.sql
```

☞ Vérifiez

Fait résultat cohérent

☞ Cette fois, faites un dump de votre base dans un fichier XML enregistré dans mes documents, le format XML étant obtenu en ajoutant l'option **--xml**. Vérifiez sous un navigateur que le fichier est bien lisible.

```
mysqldump -u root -p lemaitre_tp6urubu > backup_bdd.xml --xml
```

☞ Vérifiez **Fait**

Un lien pour découvrir toutes les options :

<https://dev.mysql.com/doc/refman/8.0/en/mysqldump.html>

III. Restauration de la base

A. En mode graphique

☞ Sous PhpMyAdmin, positionné sur votre base de données **urubu**, supprimez la table releve.

Fait

☞ Dans l'onglet "Importer", restaurez cette table par appel du script SQL de sauvegarde créé précédemment. Vérifiez l'import. **Fait**

On peut aussi n'utiliser que les instructions INSERT du script dans le cas où on perdrait uniquement le contenu de la table.

B. En mode console

☞ Connectez-vous sur le SGBD en mode console avec l'utilisateur ROOT. Indiquez que vous souhaitez travailler avec la BD urubu. : **mysql -u root -p puis use lemaître_tp6urubu**

☞ Utilisez la commande SHOW TABLES pour voir la liste des tables de la BD. **Fait**

```
MariaDB [(none)]> use urubu;
Database changed
MariaDB [urubu]> SHOW TABLES
-> ;
+-----+
| Tables_in_urubu |
+-----+
| article          |
| ccial            |
| formation        |
| noncommandes2022 |
| releve           |
| suivre           |
| zone             |
+-----+
```

☞ Supprimez la table releve. **DROP TABLE releve**

☞ Vérifiez la liste des tables de la BD. **Fait**

```
MariaDB [urubu]> SHOW TABLES;
+-----+
| Tables_in_urubu |
+-----+
| article          |
| ccial            |
| formation        |
| noncommandes2022 |
| suivre           |
| zone             |
+-----+
```

Pour restaurer les tables d'une base de données en mode console, on utilise sous l'invite de commande DOS, la commande mysql et le script SQL à restaurer en entrée tel que :

```
mysql -u nom_utilisateur nom_base < chemin/script.SQL
```

☞ Sortez de la console MySQL avec la commande exit

Fait.

☞ Sous l'interpréteur de commande CMD, restaurer la table relève à partir du script SQL précédemment créé. Indiquez la commande :

```
mysql -u root lemaitre_tp6urubu <  
C:/wamp64/bin/mariadb/mariadb10.10.2/bin/releve\_backup\_file.SQL  
Fait
```

☞ Vérifiez.

Fait.

IV. Sauvegarde automatique

Windows permet de créer des tâches et de les automatiser.

A. Préparation du script à automatiser

☞ Copier le fichier cron.php fourni dans le dossier du TP vers un nouveau dossier de www que vous nommerez sauvegardesBD.

Fait

☞ Ouvrir ce fichier et compléter les 2 zones // TODO COMPLETE WITH YOUR CONFIG :

Fait

☞ Dans la 1^{ère}, indiquer le nom de la BD, puis le nom et le mdp de l'utilisateur Eve DUPONT autoriser à sauvegarder la BD urubu. Le serveur sera localhost sans spécifier le port car le SGBD utilisé sera indiqué dans la zone // TODO COMPLETE WITH YOUR CONFIG suivante.

Fait

☞ Dans la 2^{ème} zone, modifier le chemin vers le dossier bin du SGBDR qui stocke votre BD.

Fait

☞ Observer le script puis le sauvegarder.

Fait

☞ Tester le script php. Vous devez voir apparaître le message Backing up database to suivi du nom du fichier composé en utilisant la date et l'heure de la sauvegarde pour mieux historiser les différents fichiers de sauvegardes. Vérifier également si le dossier et le fichier ont été créés.

Fait

☞ Si vous avez obtenu des erreurs, il faut les corriger (essayer par exemple avec root sans mot de passe pour savoir s'il s'agit d'un pb avec l'utilisateur).

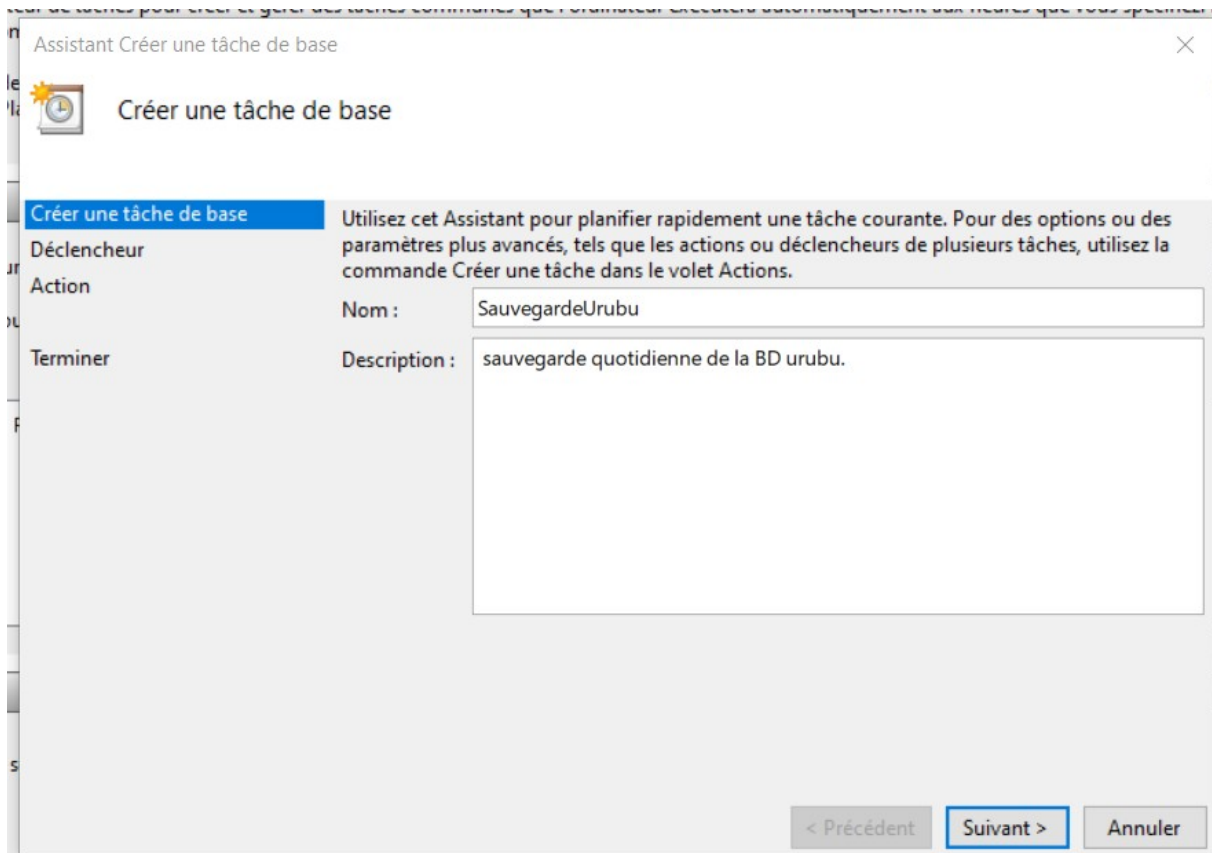
Aucune erreur

Une fois le script au point nous allons pouvoir automatiser son exécution.

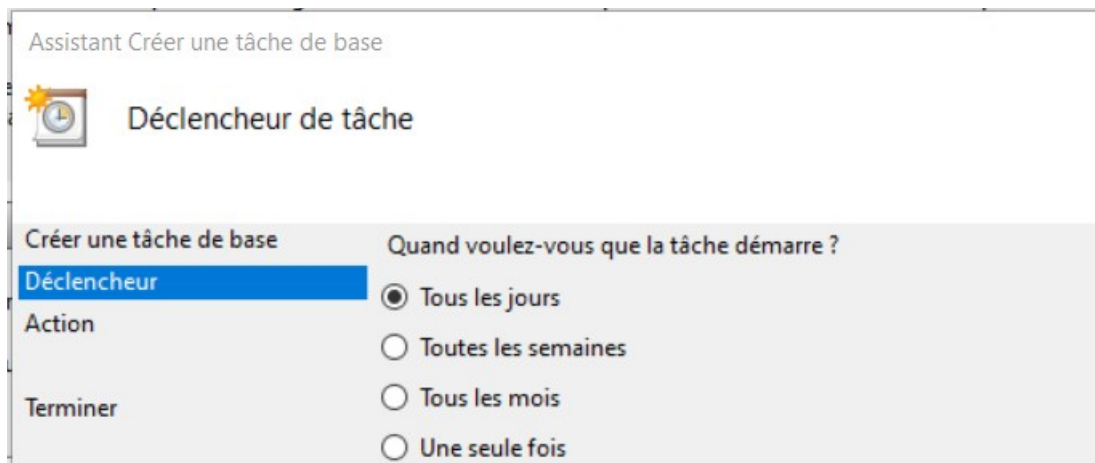
B. Outil de planification et Tâche de base Windows

1. Démarche

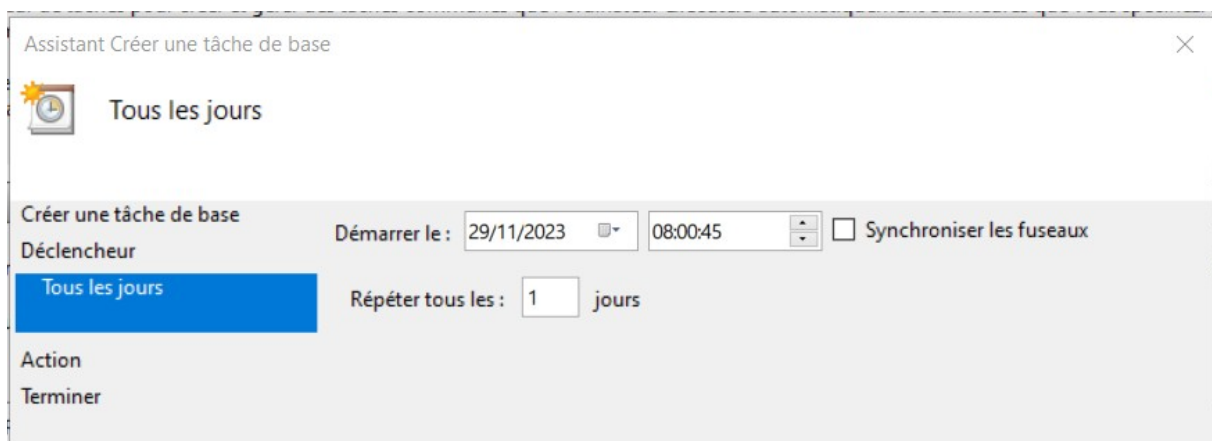
☞ Dans la rubrique Outils d'administration, ouvrir l'administrateur de tâches. Dans le volet Actions situé à droite dans la fenêtre qui s'ouvre, cliquez sur Créer une **tâche** de base. Donnez un nom à la **tâche**, **Fait**



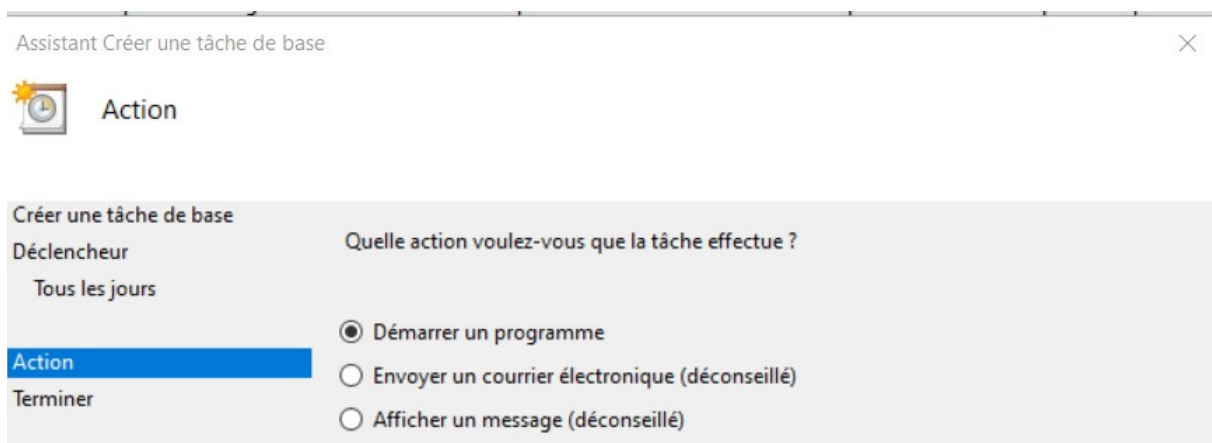
☞ Puis passez à l'écran suivant pour indiquer la fréquence : **Fait**



☞ Puis passez à l'écran suivant pour indiquer quand la tâche doit démarrer et faites la démarrer dans 15 minutes : **Fait**



☞ Puis passez à l'écran suivant pour indiquer que la tâche doit démarrer un programme : **Fait**



☞ Puis passez à l'écran suivant : **Fait**

Assistant Créer une tâche de base

 Démarrer un programme

Créer une tâche de base

Déclencheur
Tous les jours

Programme/script :
C:\wamp64\bin\php\php8.1.0\php.exe

Pargourir...

Action
Ajouter des arguments (facultatif) :
sauvegardesBD\cron.php

Démarrer un programme

Commencer dans (facultatif) :

Terminer

< Précédent Suivant > Annuler

☞ Pour indiquer qu'il s'agit d'une commande PHP : dans *programme/script* indiquer le chemin vers php.exe (voir ci-dessus). **Fait**

☞ Puis dans la zone *argument* indiquer -c puis le chemin vers le fichier php.ini puis celui vers cron.php. Pour moi l'argument sera (à adapter suivant la version de PHP) :

-c C:\wamp64\bin\php\php8.1.0\php.ini C:\wamp64\www\sauvegardesBD\cron.php

☞ Puis terminer. Vous voyez la tâche apparaître avec un statut Prêt : **Fait**

Nom	Statut	Déclencheurs	Prochaine exécution
Adobe Acrob...	Prêt	Plusieurs déclencheurs sont définis.	29/11/2023 18:00:00
GoogleUpda...	Prêt	Plusieurs déclencheurs sont définis.	29/11/2023 15:10:53
GoogleUpda...	Prêt	À 15:10 tous les jours - Après le déclenchement, recommencer tous les 1 heure pendant 1 jour.	29/11/2023 08:10:53
MicrosoftEd...	Prêt	Plusieurs déclencheurs sont définis.	29/11/2023 08:43:22
MicrosoftEd...	Prêt	À 08:13 tous les jours - Après le déclenchement, recommencer tous les 1 heure pendant 1 jour.	29/11/2023 08:13:22
MicrosoftEd...	Prêt	Plusieurs déclencheurs sont définis.	29/11/2023 08:43:21
MicrosoftEd...	Prêt	À 08:13 tous les jours - Après le déclenchement, recommencer tous les 1 heure pendant 1 jour.	29/11/2023 08:13:22
OneDrive Re...	Prêt	À 21:51 le 19/11/2023 - Après le déclenchement, recommencer tous les 1.00:00:00 indéfiniment.	29/11/2023 21:51:23
OneDrive Sta...	Prêt	À 20:00 le 01/05/1992 - Après le déclenchement, recommencer tous les 1.00:00:00 indéfiniment.	29/11/2023 20:10:41
OneDrive Sta...	Prêt	À 00:00 le 01/05/1992 - Après le déclenchement, recommencer tous les 1.00:00:00 indéfiniment.	30/11/2023 00:34:12
sauvegardes...	Prêt	À 08:36 tous les jours	29/11/2023 08:36:47

Général Déclencheurs Actions Conditions Paramètres Historique

Lorsque vous créez une tâche, vous devez spécifier l'action qui se produira au démarrage de la tâche. Pour modifier ces actions, ouvrez les pages de commande Propriétés.

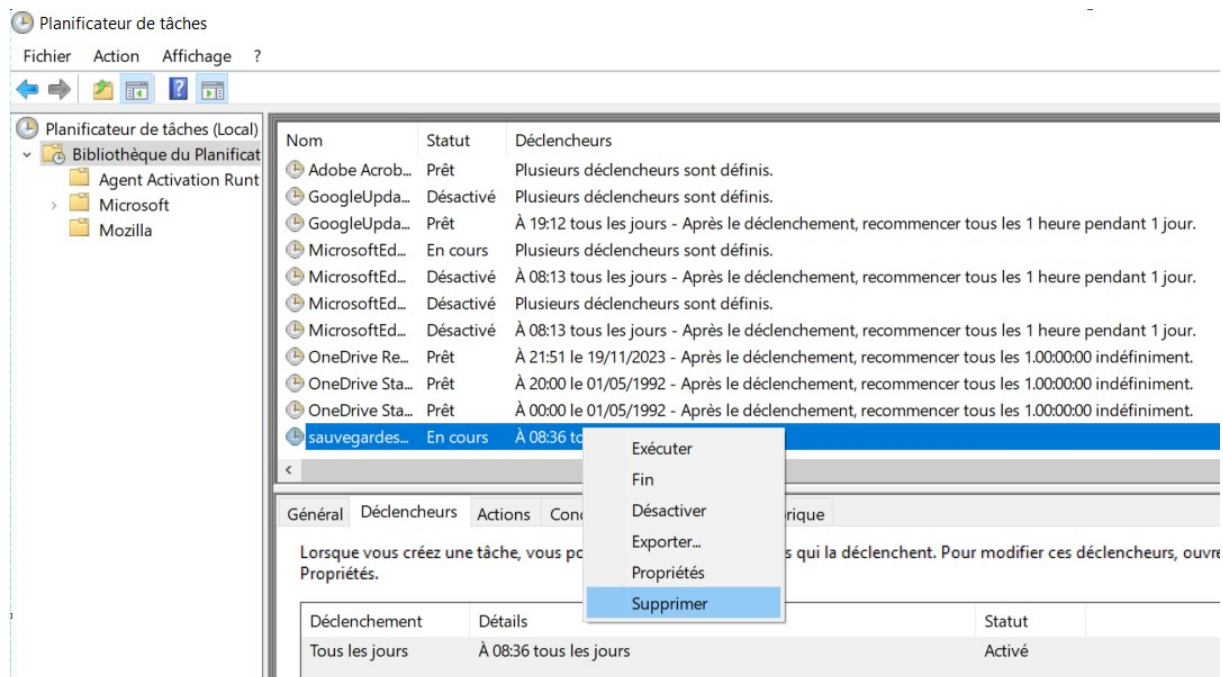
Action	Détails
Démarrer un programme	C:\wamp64\bin\php\php8.1.0\php.exe -c C:\wamp64\bin\php\php8.1.0\php.ini C:\wamp64\www\sauvegardesBD\cron.p

☞ Patienter jusqu'à l'heure de démarrage programmée.

Fait

Une fois exécutée, vous pouvez voir la dernière date d'exécution et son résultat (Opération réussie ou Erreur). **Fait.**

Grâce à un clic droit sur le nom de la tâche, on peut voir, exécuter, supprimer, modifier une tâche, en accédant à ses propriétés.



Dernière amélioration, il faut protéger l'entrée dans ce dossier (réalisable en indiquant le chemin vers localhost/sauvegardesBD). **Fait.**

2. Protection du dossier SauvegardesBD

a) Introduction

Lorsque l'on développe une application web, on en vient rapidement à se poser la question de la sécurité de certaines parties de cette dernière. En effet, certaines zones de l'application peuvent renfermer des données plus ou moins sensibles auxquelles nous ne souhaitons pas permettre la lecture (et encore moins l'écriture) à l'utilisateur lambda. Ces parties restreintes sont par exemple les *portails d'administration*, les *fichiers de configuration*, de *sauvegardes*, la *version en bêta test*...

Une méthode simple et efficace permet de sécuriser tout un dossier présent sur un serveur Web Apache au moyen d'une authentification HTTP. Cette identification sera assurée par couple un login / password.

b) Vulnérabilité du dossier

☞ Vérifier qu'on peut accéder au dossier de sauvegarde en tapant son URL.

localhost/sauvegardesBD/

Fait et fonctionne

Nous allons sécuriser l'accès à ce dossier grâce à deux fichiers de configuration qui définissent les règles de protection ainsi que les couples login / password des utilisateurs autorisés à pénétrer dans le dossier. Le premier sera le fichier *.htaccess* et le second le fichier *.htpasswd*.

c) Le fichier *.htaccess*

- Son rôle.

Le fichier *.htaccess* est un simple fichier texte qui permet de protéger l'accès au dossier dans lequel il est stocké en demandant une authentification. Il faudra le dupliquer autant de fois qu'il y a de dossiers à protéger. Comme vous pouvez le constater, il ne porte pas d'extension et commence par un point. C'est significatif d'un fichier caché.

- Son contenu.

Le code ci-dessous présente le contenu du fichier *.htaccess* qui permet de sécuriser le dossier sauvegardesBD dans c:/wamp64/www.

```
AuthName "Page d'administration protégée"
AuthType Basic
AuthUserFile "c:/wamp64/www/sauvegardesBD/.htpasswd"
Require valid-user
```

Étudions une à une les lignes de ce fichier :

Ligne 1 : C'est le texte qui invitera l'utilisateur à inscrire son login et son mot de passe.

Ligne 2 : Le type "Basic" indique la méthode qui sera utilisée pour l'authentification.

Ligne 3 : Elle indique le chemin absolu menant au fichier *.htpasswd* qui contient les couples logins / passwords autorisés à entrer dans le dossier. Ici notre site est en production donc le chemin absolu est celui du fichier sur notre serveur local (facile à indiquer). Quand le site sera hébergé chez un hébergeur, vous pourrez utiliser la fonction PHP `realpath()` qui vous indiquera le chemin absolu d'un fichier indiqué en paramètre.

☞ Créer un fichier *.htaccess* pour protéger le dossier sauvegardesBD. Indiquez ici son contenu :

Fait

```
AuthName "Page d'administration protégée"
AuthType Basic
AuthUserFile "c:/wamp64/www/sauvegardesBD/.htpasswd"
Require valid-user
```

d) Le fichier *.htpasswd*

- Son rôle

Il contient uniquement les couples logins / mots de passe des utilisateurs autorisés.

- Son emplacement et son contenu.

Dans notre exemple, nous plaçons le fichier .htpasswd à la racine du dossier /sauvegardesBD avec le fichier .htaccess. Nous aurions pu le placer ailleurs. L'important c'est de spécifier son chemin absolu sur le serveur dans le fichier .htaccess.

D'un point de vue syntaxe, chaque ligne du fichier .htpasswd doit ressembler à cela :

```
login1:motdepasse1_enClair
login2:motdepasse2_enClair
login3:motdepasse3_enClair
```

☞ Créer le fichier .htpasswd dans le dossier sauvegardesBD. Indiquez ici son contenu :

Fait

root:

☞ Tester à nouveau l'accès au dossier sauvegardesBD via son URL.

Fait

Le dossier est maintenant protégé.

3. Désactivation de la tâche

☞ Pensez maintenant à désactiver cette tâche car la sauvegarde de la BD urubu n'est pas indispensable. **Fait**

C. Planifier une tâche sans utiliser un outil

Windows nous a aidé à planifier notre script grâce à l'outil Planificateur de tâches, mais il est possible de le faire aussi en ligne de commande.

La commande Windows pour réaliser une tâche planifiée est *at*

D. Pour Linux : CRON

Pour Linux, le planificateur de tâche est le programme Cron qui permet d'exécuter automatiquement des scripts, des commandes ou des logiciels à une date et une heure spécifiée précise, ou selon un cycle défini à l'avance. D'où le nom du fichier PHP qui automatise la sauvegarde dans ce TP !

TP n°8 Personnel SQL Server

I) Introduction à SQLserver

1) Introduction

Le serveur SQLServer nommé servigBD1 est un **serveur distant hébergé sur la ferme de serveurs** auquel vous ne serez plus connecté en tant qu'administrateur.

Vous travaillerez sur des clients équipés d'un logiciel client SQLserver nommé SqlServer Management Studio qui envoie les requêtes au serveur conformément à ce schéma :

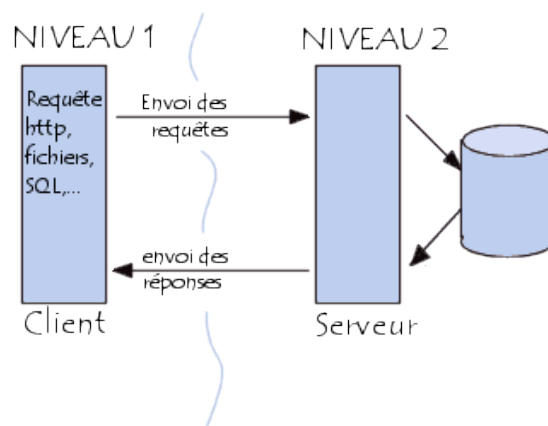


Figure 1- Architecture client serveur à 2 niveaux

2) Lancement de SQL Server

☞ Dans démarrer, exécuter, saisir ssms.exe pour SqlServer Management Studio.

ou dans démarrer>Microsoft SQLserver tools 18> Microsoft SqlServer Management Studio

Fait

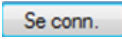
3) Connexion au serveur

☞ Indiquer comme nom de serveur servigBD1.

Fait

☞ Pour le mode d'authentification, nous avons choisi de ne pas vous donner de mots de passe **spécifiques pour SQLserver**, donc vous choisirez authentification Windows. Cela signifie que vous serez connu dans SQL Server comme INFOGESTION/login et comme membre du groupe INFOGESTION/SIO2.

Fait

☞ Dans Connection Security choisir Optional et cocher Trust server certificate, puis cliquer sur  pour vous connecter au serveur. **Fait**

Remarques

- SQLserver utilise un langage SQL nommé **TRANSACT_SQL**, les instructions de base sont celles du SQL standard mais les fonctions prédéfinies, la déclaration des variables locales, la syntaxe des IF ELSE... seront différentes de la syntaxe vue avec MySQL).
- Toute requête réalisée dans ce TP devra être reportée dans un compte-rendu.

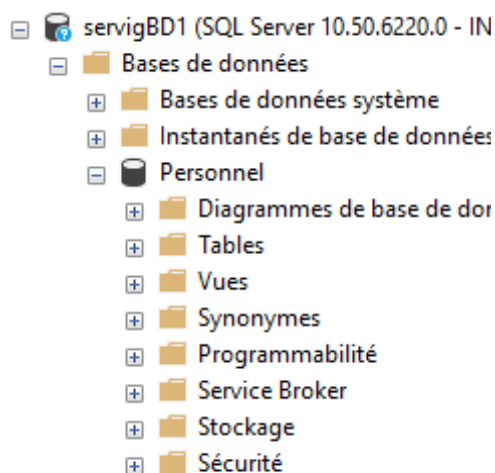
II) Travailler sur une base de données existante

1) Introduction

Pour travailler sur une base de données existante, vous devez en avoir reçu les droits par l'administrateur (database administrator) du SGBDR.

2) Choix de la base de données et requêtes

☞ Choisir la BD nommée Personnel dans le dossier Bases de données : **Fait**



Cette base de données recense les employés d'une société. Chaque employé de la société est rattaché à un département. Pour le moment, il y a 3 départements : Accounting, Research et Sales. Chaque département est identifié par un code et est localisé dans une ville (respectivement New York, Dallas, Chicago). Voici le contenu des tables :

Table employe.

	EMPNO	ENAME	JOB	HIREDATE	SAL	COM	MGR	DEPTNO
1	7369	Smith	Clerk	2011-12-17	800	NULL	7902	20
2	7499	Allen	Salesman	2012-02-20	1600	300	7698	30
3	7521	Ward	Salesman	2018-02-22	1250	500	7698	30
4	7566	Jones	Manager	2020-04-02	2975	NULL	7839	20
5	7654	Martin	Salesman	2020-09-28	1250	1400	7698	30
6	7698	Blake	Manager	2021-05-01	2850	NULL	7839	30
7	7782	Clark	Manager	2022-06-09	2450	NULL	7839	10
8	7788	Scott	Analyst	2023-11-09	3000	NULL	7566	20
9	7839	King	President	2023-11-17	5000	NULL	NULL	10
10	7844	Turner	Salesman	2024-09-08	1500	0	7698	30
11	7876	Adams	Clerk	2024-09-23	1100	NULL	7788	20
12	7900	James	Clerk	2024-12-03	950	NULL	7698	30
13	7902	Ford	Analyst	2024-12-03	3000	NULL	7566	20
14	7934	Miller	Clerk	2025-01-23	1300	NULL	7782	10

Les caractéristiques

mémorisées sont les suivantes :

- N° Employé (EMPNO)
- Nom de l'employé (ENAME)
- Emploi (JOB)
- N° du chef de l'employé (MGR)
- Date s'embauche (HIREDATE)
- Salaire (SAL)
- Commission (COM)
- Département d'affectation (DEPTNO)

La propriété MGR désigne le numéro (empno) du chef de l'employé. Exemple : Le MGR de Smith a pour numéro 7902. Il s'agit donc de Ford.

Table Departement:

DEPTNO	DEP_NOM	DEP_VILLE
10	Accounting	New York
20	Research	Dallas
30	Sales	Chicago

☞ A l'aide du contenu des tables ci-dessus, indiquez quelle contrainte pèse sur le champ MGR? **Le champ MGR est une clé étrangère qui fait référence à EMPNO de la table employe.**

Ce champ peut-il prendre n'importe-quelle valeur ? **Non, il doit correspondre à un EMPNO existant.**

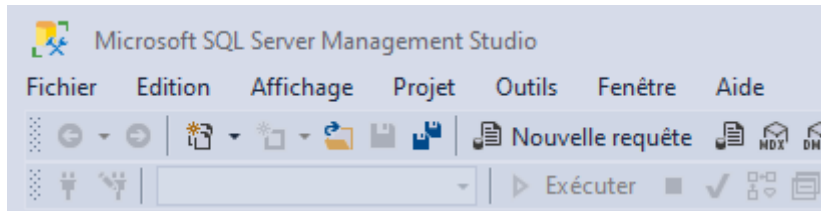
☞ Ecrire le modèle relationnel textuel correspondant à cette base de données.

Departement (DEPTNO, DEP_NOM, DEP_VILLE)
clé primaire : DEPTNO

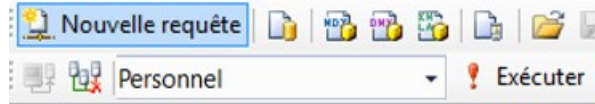
Employe (EMPNO, ENAME, JOB, HIREDATE, SAL, COM, MGR, DEPTNO)
clé primaire : EMPNO
clé étrangère : MGR qui fait référence à la clé primaire de la table Employe

3) Saisie et test d'une requête

☞ Cliquer Nouvelle requête : **Fait**



La base de données en cours est affichée



Une nouvelle fenêtre apparaît dans laquelle vous pourrez saisir et exécuter une requête.

Vous pouvez saisir votre requête, ou coller du texte, puis cliquer sur  **Exécuter** pour découvrir le résultat dans la fenêtre du bas.

Pour saisir une nouvelle requête, soit vous remplacez le texte, soit vous cliquez sur Nouvelle Requête !

☞ saisir la requête qui affiche les employés.

`SELECT * FROM Employe;`

4) Voir la structure d'une table grâce à l'interface :

☞ Développer le contenu du dossier Personnel (en cliquant sur +) puis le dossier Tables puis développer le dossier au nom de la table exemple dbo.employe (dbo=data base objects) et le dossier colonnes: vous pourrez voir tous les champs et leur type et repérer la clé primaire !

Fait

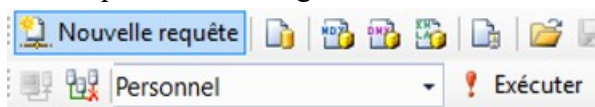
5) Voir le contenu d'une table grâce à l'interface :

☞ Pour visualiser le contenu de la table Departement, développer le contenu du dossier Personnel (en cliquant sur +) puis le dossier Tables puis clic droit sur le nom de la table à visualiser (quand vous faites un clic droit, vous utilisez un menu contextuel) et choisir 'Sélectionner les 1000 lignes du haut'.

Fait

6) Requetes sur la base de données Personnel existante :

Remarque : Vérifier régulièrement la base de données en cours est affichée



car la base de données par défaut est une base

de données système nommée Master. Si vous tentez une requête `SELECT * FROM employe;`

sur Master, vous obtiendrez le message logique :
Msg 208, Level 16, State 1, Line 1
Nom d'objet 'employe' non valide.

☞ Réaliser les requêtes répondant aux besoins suivants (à droite les résultats attendus) :

- a) Afficher les noms, jobs, salaires des 'CLERK' ainsi que des 'MANAGER' du département n°10 (une seule requête suffit).

```
SELECT ENAME, JOB, SAL, DEPTNO FROM Employe
```

```
WHERE (JOB = 'CLERK' OR JOB = 'MANAGER') AND DEPTNO = 10;
```

	ename	job	sal	deptno
1	Clark	Manager	2450	10
2	Miller	Clerk	1300	10

- b) Afficher les noms et dates d'embauche des employés embauchés avant Blake.

```
SELECT ENAME, HIREDATE FROM Employe
```

```
WHERE HIREDATE < (SELECT HIREDATE FROM Employe WHERE ENAME = 'Blake');
```

ename	hiredate
Smith	17/12/2011
Allen	20/02/2012
Ward	22/02/2018
Jones	02/04/2020
Martin	28/09/2020

- c) Afficher les noms des employés ayant le même chef que Clark.

```
SELECT ENAME AS même_manager_que_Clark FROM Employe
```

```
WHERE MGR = (SELECT MGR FROM Employe WHERE ENAME = 'Clark') AND ENAME != 'Clark';
```

même manager que Clark
Jones
Blake

- d) Indiquez pour chaque job le nombre de salariés et la moyenne des salaires.

```
SELECT JOB, COUNT(ENAME) AS nb_salariés, SUM(SAL)/COUNT(ENAME) AS Moy_salaires  
FROM Employe
```

```
GROUP BY JOB;
```

job	nb salariés	Moy salaires
Analyst	3	2433
Clerk	4	1037
Manager	3	2758
President	1	5000
Salesman	4	1400

- e) Indiquez pour chaque job ayant plus de 3 salariés, la moyenne des salaires.

```
SELECT JOB, COUNT(ENAME) AS nb_salariés, SUM(SAL)/COUNT(ENAME) AS Moy_salaires  
FROM Employe
```

```
GROUP BY JOB
```

```
HAVING COUNT(ENAME) > 3;
```

job	nb salariés	Moy salaires
Clerk	4	1037
Salesman	4	1400

- f) Afficher la somme des salaires des employés de chaque département (afficher uniquement le deptno).

```
SELECT DEPTNO, SUM(SAL) AS Somme_des_salaires FROM Employe
GROUP BY DEPTNO
```

deptno	Somme des salaires
10	8750
20	10875
30	9400
40	1300

- g) Afficher la somme des salaires des employés de chaque département (afficher le nom du département). Une ligne de résultat a disparu par rapport à la requête précédente : trouvez l'origine de ce défaut.

```
SELECT employe.DEPTNO, SUM(SAL) AS Somme_des_salaires, DEP_NOM FROM Employe
INNER JOIN Departement
```

```
ON Employe.DEPTNO = Departement.DEPTNO
```

```
GROUP BY employe.DEPTNO, DEP_NOM;
```

Une ligne a disparue car dans la table departement il n'y a pas de DEPTNO = 40.

- h) Que faudrait-il faire pour y remédier ?

Pour régler ce défaut il faut créer une contrainte de clé étrangère sur employe(DEPTNO) qui fait référence à departement(DEPTNO)

III) Créer une base de données SQLserver

1) Introduction

La base de données Personnel est une base sur laquelle vous avez des droits limités en consultation. Pour découvrir plus SQLserver, il faut maintenant créer votre propre base de données sur laquelle vous aurez des droits de dbo (database owner) qui vous permettront d'être administrateur de votre base de données et d'en découvrir plus.

Le droit de créer des bases de données a été attribué au groupe INFOGESTION/SIO2 par l'administrateur de servigbd1.

2) Créer votre base de données : devenir dbo

☞ Créer une nouvelle base de données nommée Personnel_votreNom (votre nom étant votre nom !). Pour cela, vous utiliserez l'option Nouvelle base de données du menu contextuel du dossier Bases de données.

Fait

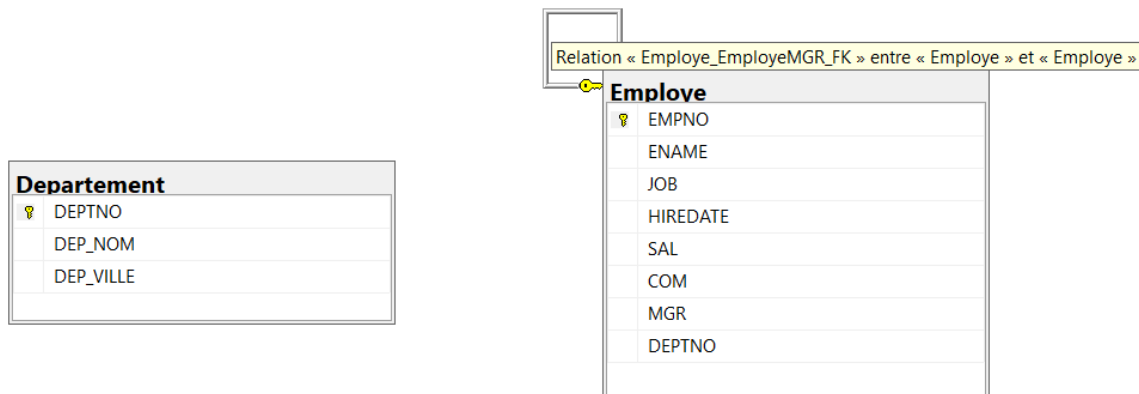
☞ Créer une nouvelle requête, puis exécuter le script creabase.sql de création de la BD.

Fait

3) Créer un schéma de base de données SQLserver

☞ dans le dossier Diagrammes de base de données de votre Base de données, choisir Nouveau diagramme. Une question vous est posée indiquant qu'il manque des objets pour créer ce diagramme : répondez OUI. Une fenêtre apparaît afin d'ajouter chaque table à placer

dans le diagramme. Avec la souris, sélectionnez toutes les tables, et cliquez sur le bouton Ajouter. Redispensez correctement les tables, vous devriez voir ce résultat : **Fait**



En survolant un lien, on peut voir le nom de la contrainte dans une info-bulle. Grâce à un clic droit sur le lien, on peut avoir un menu contextuel et choisir Propriétés pour avoir plus d'informations sur la contrainte.

☞ Observer les propriétés de l'autre lien du diagramme. Il met en œuvre l'association réflexive 'a pour chef' dont voici la représentation dans le modèle conceptuel :

Propriétés	
[Rel] Employe_EmployeMGR_FK	
(Général)	
Spécification de tables et colonnes	
Colonnes de clé primaire/unique	EMPNO
Colonnes de clés étrangères	MGR
Table de base de clé étrangère	Employe
Table de base de clé primaire/unique	Employe
Vérifier les données existantes à la création o	Oui
Concepteur de bases de données	
Appliquer la contrainte de clé étrangère	Oui
Appliquer la réplication	Oui
Spécification INSERT et UPDATE	
Identité	
(Nom)	Employe_EmployeMGR_FK
Description	

☞ Indiquer ici comment ce lien peut être mis en place dans JMerise. Coller ici le MCD:

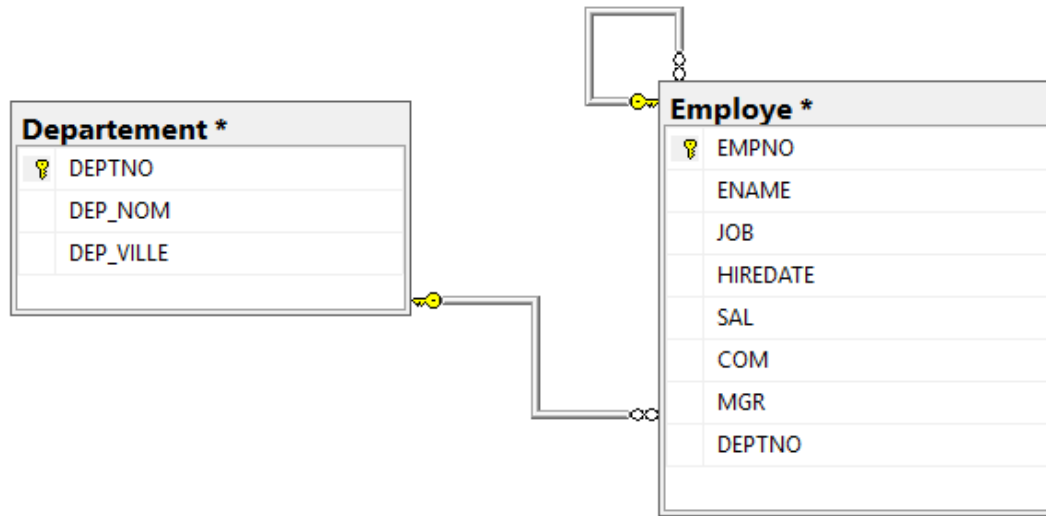
4) Travail à faire sur votre base de données :

- Rétablir la cohérence de la base de données : faire en sorte qu'on ne puisse pas assigner un employé à un département qui n'existe pas (c'est le cas pour le dernier salarié n°7935). Vous pourrez supprimer le salarié qui avait été ajouté par erreur.

Fait

DELETE FROM employe WHERE DEPTNO NOT IN (SELECT DEPTNO FROM departement)

Coller ici le diagramme de BD obtenu :



b) Créer une vue qui affiche les numéros et noms des employés avec les numéros et noms de leur chef. Tester.

```

CREATE VIEW Vue_Employes_Chefs
AS SELECT e1.EMPNO AS Num_Employe, e1.ENAME
AS Num_Employe, e2.EMPNO AS Num_Chef,
e2.ENAME AS Nom_Chef FROM Employe e1

LEFT JOIN employe e2 ON e1.MGR = e2.EMPNO;

SELECT * FROM Vue_Employes_Chefs;
    
```

Num_employe	Nom_employe	Num_chef	Nom_chef
7369	Smith	7902	Ford
7499	Allen	7698	Blake
7521	Ward	7698	Blake
7566	Jones	7839	King
7654	Martin	7698	Blake
7698	Blake	7839	King
7782	Clark	7839	King
7788	Scott	7566	Jones
7844	Turner	7698	Blake
7876	Adams	7788	Scott
7900	James	7698	Blake
7902	Ford	7566	Jones
7934	Miller	7782	Clark

c) Utiliser cette vue pour afficher le nombre d'employés de chaque chef.

```

SELECT Num_Chef, Nom_Chef, COUNT(*) AS nb_employés FROM Vue_Employes_Chefs

WHERE Num_Chef IS NOT NULL GROUP BY Num_Chef, Nom_Chef;
    
```

Num_chef	Nom_chef	nb employés
7566	Jones	2
7698	Blake	5
7782	Clark	1
7788	Scott	1
7839	King	3
7902	Ford	1

- d) Créer une vue à destination des employés qui cache les renseignements salaire et commission. Tester.

```
CREATE VIEW Vue_Employes_Sans_Salaire AS
SELECT EMPNO, ENAME, JOB, HIREDATE, DEPTNO FROM Employee;
SELECT * FROM Vue_Employes_Sans_Salaire;
```

- e) Créer une vue qui affiche la moyenne des salaires par département (avec le nom du dept). Utiliser la fonction ROUND pour arrondir le résultat. Tester la vue.

```
CREATE VIEW Vue_Moyenne_Salaires AS
SELECT d.DEP_NOM, ROUND(AVG(e.SAL), 2) AS Moyenne_Salaire
FROM Employee e
JOIN Departement d ON e.DEPTNO = d.DEPTNO
GROUP BY d.DEP_NOM;
SELECT * FROM Vue_Moyenne_Salaires;
```

num	nom	moyenne_salaires
10	Accounting	2916
20	Research	2175
30	Sales	1566

- f) Augmenter les salaires des employés du département 20 de 10% puis afficher à nouveau la moyenne des salaires par département.

```
UPDATE Employee SET SAL = SAL * 1.1 WHERE DEPTNO = 20;
```

```
SELECT * FROM Vue_Moyenne_Salaires;
```

num	nom	moyenne_salaires
10	Accounting	2916
20	Research	2392
30	Sales	1566

- g) Afficher les employés (leur numéro, nom, salaire et deptno) qui gagnent plus que la moyenne de leur département (en utilisant la vue, ce sera bien plus facile !).

```
SELECT e.EMPNO, e.ENAME, e.DEPTNO, FROM Employee e
JOIN Vue_Moyenne_Salaires v ON e.DEPTNO = (SELECT
DEPTNO FROM Departement WHERE DEP_NOM = v.DEP_NOM)
WHERE e.SAL > v.Moyenne_Salaire;
```

empno	ename	deptno	sal
7499	Allen	30	1600
7566	Jones	20	3272
7698	Blake	30	2850
7788	Scott	20	3300
7839	King	10	5000
7902	Ford	20	3300

- i) Ajouter les SIO2 aux utilisateurs de votre base de données (clic droit sur le dossier Utilisateurs du dossier Sécurité de votre BD puis Nouvel Utilisateur. Nom d'utilisateur = SIO2 et nom de connexion = INFOGESTION\SIO2.) Autoriser les étudiants de SIO2 (voir la syntaxe ci-dessous) à utiliser la vue créée en d.

```
GRANT Liste_Privilege_Objets ON Liste_objets
TO [PUBLIC / User-List]
```

[WITH GRANT OPTION] ;

WITH GRANT OPTION : si c'est spécifié, permet à l'utilisateur qui a reçu un droit de le transmettre à d'autre utilisateur à son tour.

Exemples de privilèges 'objet': SELECT, INSERT, UPDATE, DELETE, EXECUTE, ALTER ou même ALL pour accorder tous les privilèges sur un objet.

Fait

GRANT SELECT ON Vue_Moyenne_Salaires TO SIO2;

- j) Test : Demander à un camarade de tester cette autorisation. Demander lui de tester s'il a accès à la vue créée en b. pour laquelle il n'a pas d'autorisation.

Fait

- k) Créer une fonction salaireTotal() qui retourne le salaire d'un employé (sal+com).

CREATE FUNCTION salaireTotal(@empno INT)
RETURNS INT

AS

BEGIN

DECLARE @total INT;

SELECT @total = SAL + ISNULL(COM, 0) FROM Employe

WHERE EMPNO = @empno;

RETURN @total;

END;

- l)

Utiliser la fonction salaireTotal() dans une requête qui fournira le salaire total des employés du département SALES.

EMPNO	ENAME	salaireTotal
7499	Allen	1900
7521	Ward	1750
7654	Martin	2650
7698	Blake	2850
7844	Turner	1500
7900	James	950

Les employés ayant une commission NULL ne doivent pas avoir un salaireTotal NULL : si c'est le cas, il faut modifier votre fonction (vous disposez d'une fonction ISNULL()).

SELECT EMPNO, ENAME, dbo.salaireTotal(EMPNO) AS SalaireTotal

FROM Employe WHERE DEPTNO = (SELECT DEPTNO FROM Departement WHERE DEP_NOM = 'Sales');

- m) Créer une fonction anciennete() qui permet de calculer l'ancienneté d'un employé. Conseil : vous pouvez mettre d'abord au point la requête qui fait la différence entre les dates.

CREATE FUNCTION anciennete(@empno INT)
RETURNS INT AS

BEGIN

DECLARE @anciennete INT;

```

SELECT @anciennete = DATEDIFF(YEAR, HIREDATE, GETDATE()) FROM Employe
WHERE EMPNO = @empno;

RETURN @anciennete;

END

```

- n) Utiliser la fonction anciennete() dans une requête pour augmenter de 100€ la commission (com) des employés ayant plus de 15 ans d'ancienneté. Afficher le salaire total des employés du département SALES (préciser l'ancienneté pour vérifier que seuls les employés ayant plus de 15 ans d'ancienneté ont un salaire total qui a augmenté de 100€).

```

UPDATE Employe SET COM = ISNULL(COM, 0) + 100 WHERE dbo.anciennete(EMPNO) > 15;

SELECT EMPNO, ENAME, dbo.anciennete(EMPNO) AS Anciennete,
dbo.salaireTotal(EMPNO) AS SalaireTotal FROM Employe
WHERE DEPTNO = (SELECT DEPTNO FROM Departement WHERE DEP_NOM = 'Sales');

```

Evolution de la base de données :

On souhaite conserver l'historique des jobs exercés par un employé. Grâce à des requêtes SQL, mettre en place la structure de la base de données adéquate. Attention, un employé peut exercer le même job à des périodes différentes.

Création d'une table Historique_Emploi :

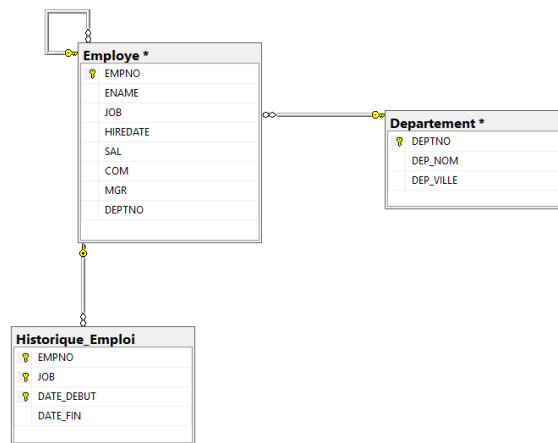
```

CREATE TABLE Historique_Emploi (
EMPNO INT,
JOB VARCHAR(50),
DATE_DEBUT DATE,
DATE_FIN DATE,
PRIMARY KEY (EMPNO, JOB, DATE_DEBUT),
FOREIGN KEY (EMPNO) REFERENCES Employe(EMPNO)
);

```

Pour qu'un employé puisse exercer le même job à des périodes différentes il faut mettre la date de début dans la clé primaire.

Coller ici le MCD proposé.



vin_num	cru_code	vin_milles	qté_tot	illes_achetées
30	MU	1992	75	
41	PF	1992	67	
50	CH	1992	75	
90	MO	1992	45	
1005	Grelet	Charles	75	
1006	Malouf	Brahim	0	

TP n°9 Bacchus

SQLserver

1) Création de la BD :

☞ Créez une BD nommée Bacchus_VotreNom grâce au menu contextuel du dossier Bases de données puis exécutez le script de création fourni. **Fait**

2) Contexte

La base de données stocke les achats de clients. Un vin appartient à un cru et un cru est associé à une région.

3) Schéma de la BD

☞ Réalisez le diagramme de base de données et collez le ici.

bacchus_lemaitre client	
PK	CLI_NUM : int(11)
	CLI_NOM : char(20)
	CLI_PRENOM : char(20)

bacchus_lemaitre acheter	
FK	CLI_NUM : int(11)
FK	VIN_NUM : int(11)
	ACH_DATE : datetime
	ACH_QTE : int(11)

bacchus_lemaitre vin	
PK	VIN_NUM : int(11)
	CRU_CODE : char(2)
	VIN_MILLES : int(11)
	VIN_QUALITE : char(1)

bacchus_lemaitre region	
PK	REG_NUM : int(11)
	REG_NOM : char(20)

bacchus_lemaitre cru	
PK	CRU_CODE : char(2)
	REG_NUM : int(11)
	CRU_NOM : char(20)

4) Autres requêtes

☞ Ecrire la requête qui permet de répondre à chacun des besoins suivants :

1. Numéros des vins avec leur qualité pour les crus suivants VO, TO, RO et MO (afficher le nom des crus).

```
SELECT 'VIN_NUM', 'VIN_QUALITE', CRU_NOM, vin.'CRU_CODE' FROM vin
INNER JOIN cru
ON vin.CRU_CODE = cru.CRU_CODE
WHERE vin.CRU_CODE = 'VO' OR vin.CRU_CODE = 'TO' OR vin.CRU_CODE =
'RO' OR vin.CRU_CODE = 'MO'
ORDER BY VIN_NUM;
```

	vin_num	vin_qualite	cru_nom	cru_code
1	10	A	VOLNAY	VO
2	11	A	VOLNAY	VO
3	20	B	TOKAY	TO
4	80	A	ROMANNEE	RO
5	85	C	ROMANNEE	RO
6	90	A	MORGON	MO

2. Liste des achats avec la quantité en litres sachant qu'une bouteille fait 0,75 l (qte_ach dans la BD est le nombre de bouteilles achetées) par les clients Julie Durand et Astrid

Dupont.

```
SELECT `CLI_NUM`, `CLI_PRENOM`, SUM(acheter.ACH_QTE*0.75) AS
nb_litres FROM `client`
INNER JOIN acheter
ON client.CLI_NUM = acheter.CLI_NUM
WHERE CLI_NUM = 'Durand' OR CLI_NUM = 'Dupont' AND
(CLI_PRENOM = 'Julie' OR CLI_PRENOM = 'Astrid')
GROUP BY `CLI_NUM`, `CLI_PRENOM`;
```

cli_nom	cli_prenom	nb_litres
Durand	Julie	56.25
Dupont	Astrid	50.25

3. Liste des clients qui n'ont rien acheté en décembre 2024 .

cli_num	cli_nom	cli_prenom
1005	Grelet	Charles
1006	Malouf	Brahim

```
SELECT * FROM `client`
WHERE `CLI_NUM` NOT IN (SELECT `CLI_NUM` FROM
acheter
WHERE ACH_DATE LIKE '2024-12-%')
```

4. Pour chaque client quel est le total de bouteilles achetées, les meilleurs clients en premier?

cli_num	cli_nom	cli_prenom	total_bouteilles_achetées
1001	Durand	Julie	75
1003	Albertini	Clémence	75
1005	Grelet	Charles	75
1002	Dupont	Astrid	67
1004	Franzin	Louis-Albert	45

```
SELECT client.`CLI_NUM`, `CLI_NUM`,
`CLI_PRENOM`, SUM(ACH_QTE) as
total_bouteilles_achetees FROM `client`
INNER JOIN acheter
ON client.CLI_NUM = acheter.CLI_NUM
GROUP BY client.CLI_NUM
ORDER BY total_bouteilles_achetees DESC
```

5. Ecrire la requête qui affiche le(s) client(s) ayant commandé le plus de bouteilles

```
SELECT client.CLI_NUM, CLI_NUM, CLI_PRENOM, SUM(ACH_QTE) as total_bouteilles FROM
CLIENT
INNER JOIN acheter ON client.CLI_NUM = acheter.CLI_NUM
GROUP BY client.CLI_NUM, CLI_NUM, CLI_PRENOM
HAVING SUM(ACH_QTE) = (SELECT MAX(total) FROM (SELECT
SUM(ACH_QTE) AS total FROM acheter GROUP BY CLI_NUM) AS
subquery);
```

cli_nom	cli_prenom	qté achetée
Durand	Julie	75
Albertini	Clémence	75
Grelet	Charles	75

5) Evolution de la base de données

1. Faites en sorte que le millésime d'un vin (VIN_MILLES), soit toujours une année inférieure à l'année en cours. Tester.


```
ALTER TABLE vin
ADD CONSTRAINT chk_millesime CHECK (VIN_MILLES < YEAR(GETDATE()));
```

- Ajouter un champ vin_stockb de type int à la table vin qui représentera le nombre de bouteilles en stock (0 valeur par défaut). Ce champ n'aura jamais une valeur négative. Visualiser le contenu de la table vin que constatez-vous ? **La colonne est null.**

```
ALTER TABLE vin
ADD vin_stockb INT DEFAULT 0 CHECK(vin_stockb >= 0);
```

La valeur par défaut à 0, ne s'appliquera que lors de l'insertion de nouvelles lignes (les lignes existantes ne sont pas concernées par la nouvelle contrainte DEFAULT 0). Un vin avec un stock négatif aurait empêché que la contrainte stock>=0 puisse être définie).

- Fixer la quantité en stock de chaque vin existant à 50 et vérifier que vos contraintes fonctionnent . Coller ici les requêtes SQL et le résultat :

```
UPDATE vin SET vin_stockb = 50;
```

	VIN_NUM	CRU_CODE	VIN_MILLES	VIN_QUALITE	vin_stockb
1	5	SA	1988	A	50
2	10	VO	1985	A	50
3	11	VO	1990	A	50
4	20	TO	1991	B	50
5	30	MU	1992	A	50
6	40	PF	1991	B	50
7	41	PF	1992	A	50
8	50	CH	1992	A	50
9	60	PO	1982	A	50
10	61	PO	1985	C	50
11	70	HB	1987	B	50
12	80	RO	1989	A	50
13	85	RO	1990	C	50
14	90	MO	1992	A	50

6) Création d'une fonction scalaire (support de cours fourni) .

☞ Créer les fonctions correspondant aux besoins suivants :

- On souhaite disposer d'une fonction qui renvoie le nombre de bouteilles achetées par un client donné. (vous avez déjà réalisé une requête du même type au 4° du paragraphe 4. Autres requêtes) . Tester pour avoir la quantité pour chaque client et vérifier que les résultats sont cohérents.

```
CREATE FUNCTION NombreBouteillesAchetees
(@CLI_NUM INT)
RETURNS INT
AS
BEGIN
DECLARE @total INT;
SELECT @total = COALESCE(SUM(ACH_QTE),0) FROM
acheter
WHERE CLI_NUM = @CLI_NUM;
RETURN @total;
END;
```

cli_num	cli_nom	cli_prenom	qté_tot
1001	Durand	Julie	75
1002	Dupont	Astrid	67
1003	Albertini	Clémence	75
1004	Franzin	Louis-Albert	45
1005	Grelet	Charles	75
1006	Malouf	Brahim	0

```
SELECT cli_num, cli_nom, cli_prenom, dbo.NombreBouteillesAchetees(CLI_NUM) FROM
client;
```

2. On souhaite disposer d'une fonction qui retourne la réduction pour un client. La réduction obtenue sera utilisée pour effectuer une remise qui permettra de fidéliser le client. La réduction est de:

- . 0 % si le client a acheté moins de 45 bouteilles ;
- . 5 % si le client a acheté entre 45 et 70 bouteilles ;
- . 10 % si le client a acheté plus de 70 bouteilles.

Créer et tester votre fonction.

```
CREATE FUNCTION ReductionClients (@client_id INT)
RETURNS INT
AS
BEGIN
DECLARE @nbBouteilles INT;
DECLARE @reduction INT;
SET @nbBouteilles = dbo.NombreBouteillesAchetees(@client_id);
IF @nbBouteilles < 45
SET @reduction = 0;
ELSE IF @nbBouteilles BETWEEN 45 AND 70
SET @reduction = 5;
ELSE IF @nbBouteilles > 70
SET @reduction = 10;
RETURN @reduction
END;

SELECT cli_num, dbo.ReductionClients (cli_num) FROM acheter;
```

7) Procédures stockées (support de cours fourni)

☞ Créer une procédure stockée entrer_stock() qui permettra d'augmenter la quantité en stock d'un produit donné de la quantité souhaitée (cette procédure sera exécutée pour chaque produit reçu lors d'une livraison).

```
CREATE PROCEDURE entrer_stock (@vin_num INT, @quantite INT)
AS
BEGIN
UPDATE VIN
SET vin_stockb = vin_stockb + @quantite
WHERE VIN_NUM = @vin_num;
END;
```

☞ Tester.

```
EXEC dbo.entrer_stock 5, 20;
SELECT * FROM vin;
```

	VIN_NUM	CRU_CODE	VIN_MILLES	VIN_QUALITE	vin_stockb
1	5	SA	1988	A	70
2	10	VO	1985	A	50
3	11	VO	1990	A	50
4	20	TO	1991	B	50
5	30	MU	1992	A	50
6	40	PF	1991	B	50
7	41	PF	1992	A	50
8	50	CH	1992	A	50
9	60	PO	1982	A	50
10	61	PO	1985	C	50
11	70	HB	1987	B	50
12	80	RO	1989	A	50
13	85	RO	1990	C	50
14	90	MO	1992	A	50

8) Triggers (support de cours fourni)

☞ Créer un trigger qui annule un achat d'un vin, si sa quantité en stock est insuffisante.

```
CREATE TRIGGER AnnulerAchatStock
ON ACHETER
AFTER INSERT
AS BEGIN
IF EXISTS (SELECT * FROM ACHETER a JOIN VIN v ON a.VIN_NUM = v.VIN_NUM WHERE a.ACH_QTE
> v.vin_stockb)
BEGIN RAISERROR ('Stock insuffisant', 16, 1);
ROLLBACK TRANSACTION;
END;
END;
```

Pour les plus avancés uniquement :

☞ créer le trigger qui gère la modification de la quantité dans la table Vin.

```
CREATE TRIGGER MajStockVin
ON ACHETER
AFTER INSERT
AS
BEGIN
UPDATE VIN
SET vin_stockb = vin_stockb + a.ACH_QTE
FROM VIN v
INNER JOIN ACHETER a ON v.VIN_NUM = a.VIN_NUM;
END;
```

9) Fonctions qui retournent des tables

SQL server permet de créer des fonctions qui retournent des tables.

Elles acceptent des paramètres et retournent des lignes de résultats exploitables comme une table.

Exemple : Pour retourner les vins d'un millésime :

```
CREATE FUNCTION VinsMille (
    @year INT
)
RETURNS TABLE
AS
RETURN
    SELECT
        vin_num,
        cru_code,
        vin_milles
    FROM
        vin
    WHERE
        vin_milles = @year;
```

Test :

```
SELECT * FROM dbo.VinsMille(1992);
```

vin_num	cru_code	vin_milles
30	MU	1992
41	PF	1992
50	CH	1992
90	MO	1992

A vous de jouer :

☞ Créer une fonction pour retourner les vins dont la quantité en stock est inférieure à un seuil (vous vérifierez et adapterez les quantités existantes avant de tester). Tester pour voir les vins qui ont un stock inférieur à 50 bouteilles.

```
CREATE FUNCTION VinStockSeuil (@seuil INT)
RETURNS TABLE
AS
RETURN SELECT vin_num, vin_stockb FROM vin WHERE vin_stockb < @seuil;

SELECT * FROM dbo.VinStockSeuil(50);
```

TP n°9 Bacchus suite : les autorisations

1) Ajout de connexions au niveau de la BD :

Pour réaliser ce TP, et tester les autorisations attribuées à d'autres, il faut créer des utilisateurs supplémentaires de votre BD.

Les utilisateurs que vous allez ajouter sont : INFOGESTION\SIO2 et INFOGESTION\testSIO2Slam : le premier représente le groupe windows SIO2 du domaine infogestion, le second sera votre voisin de table, ou par défaut un utilisateur testSIO2slam.

Si vous utilisez testSIO2Slam, il vous faut un autre PC sur lequel j'ouvrirai une session en tant que testSIO2Slam.

Pour pouvoir ajouter ces utilisateurs au niveau de votre dossier BD pour leur donner des droits, il faut d'abord que ces utilisateurs soient autorisés à se connecter au SGBDR servigBD1. Pour cela, un SYSADMIN (administrateur système, ou parle aussi de dba pour database administrator) de SQL Server, a ajouté deux connexions au niveau du dossier Sécurité/connexions du SGBDR servigBD1 : ce qui donne le droit aux utilisateurs de se connecter au serveur et d'appartenir à un groupe par défaut nommé PUBLIC.

Pour rappel, le SYSADMIN a aussi autorisé le groupe INFOGESTION\SIO2 à créer des bases de données (en lui associant les droits de *dbcreator* qui est un rôle de serveur). SQLServer fournit 9 rôles de serveurs au SYSADMIN qui a l'exception de PUBLIC sont des rôles de serveurs fixes (leurs droits ne peuvent pas être modifiés).

Le SYSADMIN intervient au niveau de la gestion du SGBDR. Dans ce TP, vous intervenez en tant que dbo (data base Owner) : ce qui vous permet de pouvoir donner des droits sur les bases de données dont vous êtes propriétaire.

☞ Dans un premier temps, il faut ajouter le groupe INFOGESTION\SIO2 : dans le dossier *sécurité*, *utilisateurs* de votre base de données, clic droit puis *nouvel connexion*.

- Le nom d'utilisateur est le nom que possèdera l'utilisateur dans SQLServer au niveau de votre base de données.
- Le nom d'accès est soit le nom de connexion Windows de l'utilisateur donc ici INFOGESTION\SIO2 soit celui de sqlServer (quand l'utilisateur possède un identifiant de connexion propre à SQLserver).

☞ Ajouter également l'utilisateur INFOGESTION\xxx de votre choix (votre voisin de table par exemple), que je l'appellerai testSIO2Slam dans la section travail à faire. **Fait**

Travail à faire :

Rédiger un compte-rendu sur la mise en place des autorisations suivantes et leurs tests (pour les tests, il faut utiliser un PC où vous aurez ouvert une session en tant que testSIO2Slam ou demander à votre voisin de table de faire les tests).

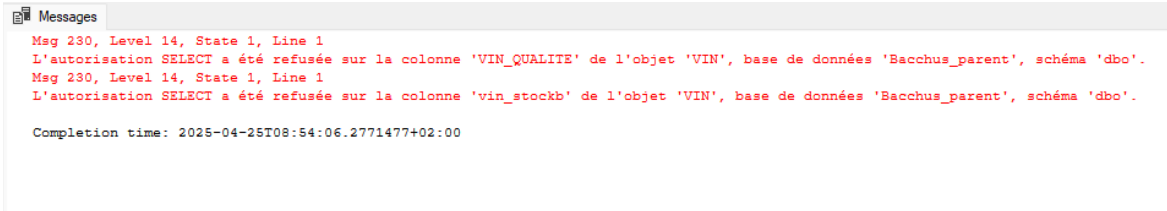
Un cours sur les Autorisations SQLserver vous est fourni.

- Donner la permission à testSIO2Slam de réaliser des requêtes SELECT sur la table *vin* de votre base de données *Bacchus*. Pour des raisons de confidentialité, seules les colonnes *vin_num*, *vin_milles* et *cru_code* seront accessibles.

```
GRANT SELECT (vin_num, vin_milles, cru_code) ON dbo.VIN TO yannis
```

Pensez à vérifier que tous les champs de la table ne sont pas accessibles.

```
SELECT * FROM VIN
```



- Réaliser un DENY pour interdire au groupe SIO2 de réaliser un SELECT sur la table Vin.

```
DENY SELECT ON dbo.VIN TO [INFOGESTION\SIO2];
```

Est-ce que testSIO2Slam bénéficie toujours du droit accordé au 1 ? **NON.**

Pourquoi d'après vous ?

- Créer une vue *vin_qA* qui affiche les vins de qualité A (on devra voir les informations : numero, millésime et cru).

```
CREATE VIEW dbo.vin_qA AS
SELECT vin_num, vin_milles, cru_code
FROM dbo.VIN
WHERE VIN_QUALITE = 'A';
```

Autoriser testSIO2Slam à réaliser un SELECT sur cette vue.

```
GRANT SELECT ON dbo.vin_qA TO yannis;
```

Tester Est-ce possible malgré le DENY précédent ? **Oui**

```
SELECT * FROM dbo.vin_qA
```

Rappel la réalisation des vues et des autorisations uniquement sur les vues sont un atout quant à la sécurité des données.

- Modifier la vue : ajouter la qualité du vin. testSIO2Slam peut-il voir les informations de la vue ?

```
ALTER VIEW dbo.vin_qA AS
SELECT vin_num, vin_milles, cru_code, vin_qualite
FROM dbo.VIN
WHERE vin_qualite = 'A';
```

- testSIO2Slam peut-il insérer des enregistrements à l'aide de cette vue ? **Non**

Si ce n'est pas le cas, autorisez-le.

```
GRANT INSERT ON dbo.vin_qA TO yannis;
```

- Modifier la vue : ajouter un WITH CHECK OPTION.

```
ALTER VIEW dbo.vin_qA AS
SELECT vin_num, vin_milles, cru_code, vin_qualite
FROM dbo.VIN
WHERE vin_qualite = 'A'
WITH CHECK OPTION;
```

7. testSIO2Slam peut-il utiliser la fonction nb_bouteilles_achetees() pour voir le nombre de bouteilles achetées par le client n°1001 (le nom de la fonction est à préfixer par dbo). Si ce n'est pas le cas, autorisez-le.

```
GRANT EXECUTE ON dbo.NombreBouteillesAchetees TO yannis;
```

8. testSIO2Slam peut-il voir le nombre de bouteilles achetées par chaque client ?
Non.

Pourquoi ?

La fonction est limitée pour voir le nombre de bouteilles achetées par un seul client.

9. testSIO2Slam peut-il utiliser entrerStock ?
Oui.

Donner le droit au groupe SIO2.

```
GRANT EXECUTE ON dbo.entrer_stock TO [INFOGESTION\SIO2];
```

Il devra enregistrer une entrée en stock de 50 bouteilles pour le vin n°5.

```
EXEC dbo.entrer_stock @vin_num = 5, @quantite = 50;
```

10. Interdire à testSIO2Slam d'utiliser entrerStock.

```
DENY EXECUTE ON dbo.entrer_stock TO yannis;
```

☞ Peut-on donner des droits sur les triggers ?

Non

☞ Retrouver dans l'interface de Management Studio, l'endroit où on voit que testSIO2Slam ne peut pas utiliser entrerStock.

Aller dans Base de donnée → Sécurité → Utilisateurs → yannis

Clic droit → Propriétés

Onglet Securables → Chercher entrer_stock

Colonne Permissions → vous verrez un DENY explicite